HTML

Índice

1 Introducción a HTML 1

- 1.1 Estructura Básica 8
- 1.2 Atributos Globales 9
- 1.3 Comentarios en HTML 11
- 1.4 Sintaxis Básica de Elementos HTML 11
- 1.5 Buenas Prácticas 12
- 1.6 Validación 13
- 1.7 Organización de Archivos 13
- 1.8 Herramientas de Desarrollo (DevTools) 14
- 1.9 Ejercicios Prácticos 14
- 1.10 Soluciones 16

2 Metadatos en HTML 18

- 2.1 El Elemento head 18
- 2.2 Elementos Principales del head 19
- 2.3 Metadatos para Redes Sociales: Open Graph y Twitter Cards 23
- 2.4 Mejores Prácticas 24
- 2.5 Ejercicios 25
- 2.6 Soluciones 26
- 2.7 Referencias y Recursos 27

3 Elementos de Texto en HTML 29

- 3.1 Estructura Fundamental del Texto 29
- 3.2 Semántica y Formato de Texto en Línea 31
- 3.3 Citas, Referencias y Definiciones 33
- 3.4 Código y Texto Preformateado 33

3.5 Entidades HTML (Caracteres Especiales) 34
3.6 Mejores Prácticas 35
3.7 Ejercicios 35
3.8 Soluciones 37
3.9 Recursos 38

4 Listas en HTML 39

4.1 Listas Ordenadas (ol) 39
4.2 Listas No Ordenadas (ul) 41
4.3 Listas de Definición (dl) 42
4.4 Listas Anidadas 42
4.5 Casos de Uso Comunes 44
4.6 Ejercicios 45
4.7 Soluciones 46
4.8 Referencias y Recursos 47

5 Enlaces en HTML 48

5.1 Sintaxis Básica 48
5.2 Tipos de Enlaces 48
5.3 Rutas de Enlaces 49
5.4 Atributos de Enlaces 52
5.5 Enlaces Internos en la Página 53
5.6 Mejores Prácticas 54
5.7 Ejercicios 55
5.8 Soluciones 57
5.9 Recursos 58

6 Tablas en HTML 59

6.1 Estructura Básica 59
6.2 Secciones de Tabla 61
6.3 Spanning y Grupos 62
6.4 Tablas Responsivas 64
6.5 Mejores Prácticas 65

6.6 Atributos Útiles para Accesibilidad 67
6.7 Ejercicios 68
6.8 Soluciones 70
6.9 Referencias y Recursos 72

7 Multimedia y Contenido Embebido 73

7.1 Imágenes 73
7.2 Audio y Video 76
7.3 Contenido Embebido con IFrames 78
7.4 Mejores Prácticas Generales 80
7.5 Alternativas Modernas a los IFrames 81
7.6 Ejercicios 81
7.7 Soluciones 82
7.8 Referencias y Recursos 84

8 Optimización del Rendimiento 85

8.1 Optimización de Imágenes 85
8.2 Optimización de Contenido Embebido 87
8.3 Optimización de la Carga de Recursos (CSS y JS) 88
8.4 Ejercicios 89
8.5 Soluciones 91
8.6 Referencias y Recursos 92

9 Formularios 93

9.1 Estructura de un Formulario HTML 93
9.2 Controles de Formulario: Recopilando Datos del Usuario 95
9.3 Validación de Formularios 103
9.4 Eventos de Formulario 106
9.5 Mejores Prácticas 107
9.6 Ejercicios 107
9.7 Soluciones 110

9.8 Referencias y Recursos 112

10 HTML Semántico: Estructura con Significado 113

10.1 Elementos Semánticos Principales 113
10.2 Estructura Semántica Completa 117
10.3 Elementos Semánticos Adicionales 118
10.4 Mejores Prácticas 119
10.5 Beneficios del HTML Semántico 120
10.6 Ejercicios 120
10.7 Soluciones 123
10.8 Referencias y Recursos 125

11 Accesibilidad en HTML 126

11.1 Principios WCAG 126
11.2 ARIA Roles y Atributos 128
11.3 Navegación por Teclado 130
11.4 Mejores Prácticas 132
11.5 Lista de Verificación de Accesibilidad 135
11.6 Ejercicios 136
11.7 Soluciones 137
11.8 Referencias y Recursos 138

12 Elementos Interactivos en HTML 140

12.1 details y summary 140
12.2 dialog 141
12.3 canvas 142
12.4 Drag and Drop (Arrastrar y Soltar) 144
12.5 Mejores Prácticas 146
12.6 Ejercicios 147
12.7 Soluciones 148

12.8 Referencias y Recursos 149

13 Atributos data-* en HTML 150

13.1 Sintaxis Básica 150
13.2 La API dataset 151
13.3 Uso con CSS 152
13.4 Casos de Uso Prácticos 154
13.5 Mejores Prácticas 156
13.6 Ejercicios 156
13.7 Soluciones 158
13.8 Referencias y Recursos 159

14 Introducción a Web Components 160

14.1 Bloques de Construcción Nativos 160
14.2 Ejemplo Práctico: Creando un Elemento mi-tarjeta 161
14.3 ¿Por Qué es Importante? 162
14.4 Ejercicios 163
14.5 Soluciones 165
14.6 Referencias y Recursos 166

15 Ejercicios Generales de HTML 167

15.1 Ejercicios 167
15.2 Soluciones Orientativas 170
15.3 Recursos 173

1 Introducción a HTML

HTML, siglas de *HyperText Markup Language*, es el lenguaje de marcado que constituye la base de la World Wide Web. Fue creado a principios de la década de 1990 por Tim Berners-Lee como parte de un sistema para compartir documentos científicos mediante hipertexto. Desde entonces, ha evolucionado hasta convertirse en el estándar fundamental para estructurar el contenido de la mayoría de sitios web.

> A diferencia de los lenguajes de programación, **HTML no describe procesos ni cálculos**. Su función principal es establecer la estructura y el significado del contenido: define qué es un título, un párrafo, una imagen, una lista, una tabla o un formulario, y cómo se relacionan esos elementos entre sí dentro de una página web. Esto permite a los navegadores interpretar y representar dicho contenido de forma coherente para los usuarios.

Desde su versión original, HTML ha pasado por distintas etapas de estandarización. Tras una fase inicial dominada por extensiones no oficiales de los navegadores, el lenguaje fue formalizado por el W3C (World Wide Web Consortium). En la actualidad, HTML5 es la versión vigente y está en continuo desarrollo bajo la especificación del WHATWG (Web Hypertext Application Technology Working Group), con un enfoque centrado en la interoperabilidad, la accesibilidad y el soporte para aplicaciones web modernas.

HTML5 introdujo mejoras importantes, como nuevos elementos semánticos (`<article>`, `<section>`, `<nav>`, `<header>`, `<footer>`), soporte nativo para audio y vídeo, mejoras en formularios y nuevas API que permiten integrar capacidades interactivas directamente desde el navegador, sin necesidad de tecnologías externas como Flash.

Dominar HTML es un requisito esencial para cualquier persona que desee trabajar en el desarrollo web o comprender cómo funciona Internet desde el punto de vista técnico. Aunque muchas herramientas modernas generan HTML automáticamente, comprender el lenguaje permite tener control real sobre el contenido, la accesibilidad, la estructura y la calidad de una página web.

Este conocimiento no solo es útil para desarrolladores, sino también para diseñadores, editores digitales, responsables de SEO, especialistas en accesibilidad y cualquier profesional relacionado con la creación o mantenimiento de contenidos en la web.

1.1 Estructura Básica

Todo documento HTML tiene una estructura básica que consiste en:

HTML

```html
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Mi primera página web</title>
  </head>
  <body>
    <!-- Aquí va el contenido visible -->
  </body>
</html>
```

Vamos a explicar cada parte:

1.1.1 DOCTYPE

- `<!DOCTYPE html>` indica que estamos usando HTML5
- Es obligatorio y debe ser lo primero en el documento
- No distingue entre mayúsculas y minúsculas

1.1.2 Elemento html

- Es la raíz de todo documento HTML
- El atributo `lang` especifica el idioma del contenido
- Contiene todo el contenido de la página

1.1.3 Elementos head y body

- `<head>` : Contiene información sobre el documento
- `<body>` : Contiene todo el contenido visible

1.2 Atributos Globales

Los atributos globales son aquellos que pueden usarse en cualquier elemento HTML. Los más comunes son:

1.2.1 class

- Define una o varias clases para un elemento
- Se usa principalmente para CSS y JavaScript

HTML

```html
<div class="contenedor principal">
```

1.2.2 id

- Define un identificador único
- No debe repetirse en la página
- Se usa para CSS, JavaScript y enlaces internos

HTML

```html
<div id="seccion-principal">
```

1.2.3 style

- Define estilos CSS directamente en el elemento

HTML

```
<p style="color: blue; font-size: 16px;">
```

Aunque el atributo `style` existe, en el desarrollo profesional **se considera una mala práctica** usarlo para dar diseño a la web. Hace que el código sea difícil de mantener y sucio. **La regla de oro:** El HTML es para la **estructura** y el contenido. El CSS (en un archivo separado) es para el **diseño** y la apariencia.

1.2.4 title

- Define información adicional que se muestra al pasar el ratón

HTML

```
<p title="Esto es un tooltip">Pasa el ratón por aquí</p>
```

1.2.5 data-*

- Permite almacenar información personalizada
- Debe empezar con "data-"

HTML

```
<div data-usuario="juan" data-id="123">
```

1.3 Comentarios en HTML

Los comentarios son notas que se pueden incluir en el código y no se muestran en la página:

HTML

```
<!-- Esto es un comentario -->
<!--
  También pueden ser
  comentarios de
  múltiples líneas
-->
```

1.4 Sintaxis Básica de Elementos HTML

1.4.1 Elementos con contenido

HTML

```
<etiqueta>contenido</etiqueta>
```

1.4.2 Elementos vacíos

HTML

```
<etiqueta/>
<!-- o -->
<etiqueta>
```

1.4.3 Elementos con atributos

HTML

```
<etiqueta atributo="valor">contenido</etiqueta>
```

1.5 Buenas Prácticas

1. **Indentación**: Usa una indentación consistente para hacer el código más legible

HTML

```
<div>
  <p>Este texto está bien indentado</p>
</div>
```

2. **Minúsculas**: Usa minúsculas para nombres de elementos y atributos

HTML

```
<!-- Bien -->
<div class="contenedor">
<!-- Evitar -->
<DIV CLASS="contenedor">
```

3. **Comillas**: Usa siempre comillas para los valores de los atributos

HTML

```
<!-- Bien -->
<img src="imagen.jpg" alt="Mi imagen">
<!-- Evitar -->
<img src=imagen.jpg alt=Mi imagen>
```

1. **Cerrar elementos**: Cierra todos los elementos correctamente

HTML

```html
<!-- Bien -->
<p>Texto</p>
<!-- Evitar -->
<p>Texto
```

1.6 Validación

Es importante validar el código HTML para asegurarse de que está bien formado. Puedes usar:

- El validador del W3C: https://validator.w3.org/
- Extensiones de tu editor de código
- Herramientas de desarrollo del navegador

1.7 Organización de Archivos

Para mantener un proyecto ordenado, los desarrolladores siguen una estructura estándar de carpetas. No tires todos tus archivos en un solo lugar.

Estructura recomendada:

```
mi-proyecto/
├── index.html          # Tu página principal siempre debe llamarse index.html
├── acerca-de.html      # Otras páginas HTML al mismo nivel
├── css/                # Carpeta para hojas de estilo
│   └── estilos.css
├── js/                 # Carpeta para scripts
│   └── main.js
└── assets/             # Carpeta para recursos multimedia
    ├── images/
    │   └── logo.png
    └── docs/
        └── manual.pdf
```

1.8 Herramientas de Desarrollo (DevTools)

No estás programando a ciegas. Todos los navegadores modernos incluyen un inspector de código.

1.8.1 Cómo usarlo

1. Haz clic derecho en cualquier elemento de una página web.
2. Selecciona **"Inspeccionar"** (o "Inspect").

Se abrirá un panel donde verás:

- **Elementos (DOM):** El HTML tal cual lo está interpretando el navegador ahora mismo (útil para ver si cerraste mal una etiqueta).
- **Estilos:** El CSS que se está aplicando.
- **Consola:** Donde aparecen los errores de JavaScript.

¿Por qué usarlo? Si tu imagen no carga o tu tabla se ve rota, no adivines. Inspecciona el elemento. A menudo verás que la ruta de la imagen está mal o que el navegador cerró una etiqueta `</div>` donde no debía porque se te olvidó cerrarla tú.

1.9 Ejercicios Prácticos

A continuación, se presentan algunos ejercicios para que pongas en práctica los conceptos básicos de HTML que has aprendido. Intenta resolverlos por tu cuenta antes de ver las soluciones.

1.9.1 Ejercicio 1: Estructura Básica

Crea la estructura de un documento HTML5 válido. El título de la página debe ser **"Mi Página de Ejercicios"** y el idioma del documento debe ser español. Dentro del `<body>`, añade un comentario que diga **"Aquí va el contenido visible de mi página"**.

1.9.2 Ejercicio 2: Elementos y Atributos Globales

Dentro del `<body>` del ejercicio anterior, crea un párrafo (`<p>`) con el siguiente contenido: "**Este es mi primer ejercicio de HTML**". Asígnale un identificador único con el valor **`parrafo-principal`** y una clase con el valor **`texto-destacado`**. Finalmente, añade un `title` (tooltip) que muestre el texto "**Párrafo de introducción**" al pasar el ratón por encima.

1.9.3 Ejercicio 3: Atributos `data-*` y Comentarios

Crea un elemento `<div>` que funcione como un contenedor. Dentro de este `div`, añade un atributo personalizado `data-section-id` con el valor "**001**". Justo encima del `div`, escribe un comentario en HTML que diga: "**Inicio de la sección principal**".

1.9.4 Ejercicio 4: Corregir Código (Buenas Prácticas)

El siguiente fragmento de código HTML no sigue las buenas prácticas. Corrígelo para que esté bien indentado, use minúsculas para etiquetas y atributos, utilice comillas en los valores de los atributos y cierre correctamente todos los elementos.

HTML

```html
<DIV CLASS=CONTENEDOR>
    <P style=color:red>Este es un texto importante
<SPAN>Y esto es una continuación.</SPAN>
</DIV>
```

1.10 Soluciones

Aquí puedes comprobar tus respuestas.

1.10.1 Solución Ejercicio 1

HTML

```html
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>Mi Página de Ejercicios</title>
  </head>
  <body>
    </body>
</html>
```

1.10.2 Solución Ejercicio 2

HTML

```html
<p id="parrafo-principal" class="texto-destacado" title="Párrafo de introducción">Este es mi
primer ejercicio de HTML.</p>
```

1.10.3 Solución Ejercicio 3

HTML

```html
<!-- Inicio de la sección principal -->
<div data-section-id="001">
  </div>
```

1.10.4 Solución Ejercicio 4

HTML

```html
<div class="contenedor">
  <p style="color: red;">Este es un texto importante.</p>
  <span>Y esto es una continuación.</span>
</div>
```

2 Metadatos en HTML

Los **metadatos**, que literalmente significan "datos sobre datos", son instrucciones y descripciones que incluimos en un documento HTML para que las máquinas puedan entenderlo mejor. Aunque son invisibles para un visitante que navega por la página, son fundamentales para los **navegadores web**, los **motores de búsqueda** (como Google) y las **plataformas de redes sociales**.

Piensa en ellos como la etiqueta de información de un producto en un supermercado: no es el producto en sí, pero te dice qué es, de qué está hecho y cómo usarlo. De la misma manera, los metadatos le dicen a los navegadores cómo mostrar tu página, a los buscadores de qué trata para clasificarla, y a las redes sociales cómo presentarla cuando alguien la comparte.

2.1 El Elemento head

El elemento `<head>` es el contenedor donde se alojan todos los metadatos de un documento HTML. Nada de lo que coloques dentro del `<head>` se renderizará directamente en la ventana principal del navegador, pero su contenido es vital para que la página funcione correctamente.

Sus funciones principales son:

- **Definir información clave sobre el documento**: Aquí se especifica el título de la página, la codificación de caracteres para que los textos se vean bien, y la descripción que aparecerá en los resultados de búsqueda.

- **Enlazar recursos externos**: Es el lugar para conectar archivos que la página necesita para funcionar o verse bien, como hojas de estilo CSS, fuentes personalizadas o el pequeño icono (favicon) que aparece en la pestaña del navegador.
- **Configurar el SEO (Search Engine Optimization)**: A través de etiquetas específicas, damos instrucciones a los robots de los buscadores sobre cómo deben indexar y clasificar nuestro contenido.
- **Establecer el título de la página**: Define el texto que se muestra en la parte superior de la ventana o pestaña del navegador y que es usado por los buscadores como el título principal en sus resultados.

HTML

```html
<head>
  <meta charset="UTF-8">

  <link rel="icon" type="image/png" href="favicon.png">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Mi Página Web | Un lugar para aprender</title>

  <meta name="description" content="Aprende sobre la importancia de los metadatos en HTML y cómo usarlos para mejorar tu sitio web.">

  <link rel="stylesheet" href="estilos.css">
</head>
```

2.2 Elementos Principales del head

2.2.1 La etiqueta `<title>`

Es la única etiqueta **obligatoria** dentro del `<head>`. Su contenido es crucial tanto para la experiencia del usuario (UX) como para el SEO.

- **Navegador**: Aparece en la pestaña, permitiendo a los usuarios identificar tu página fácilmente entre muchas otras.
- **SEO**: Los motores de búsqueda lo utilizan como el titular principal en sus páginas de resultados. Un buen título es descriptivo, conciso y atractivo para incentivar los clics.

HTML

```html
<title>Mi Página Web - Inicio</title>
```

2.2.2 Las Etiquetas `<meta>` (Meta Tags)

Las etiquetas `<meta>` son muy versátiles y proporcionan diferentes tipos de metadatos.

2.2.2.1 Codificación de caracteres (`charset`)

Esta etiqueta le dice al navegador qué conjunto de caracteres debe usar para interpretar el texto del documento. `UTF-8` es el estándar universal hoy en día porque es capaz de representar prácticamente cualquier carácter o símbolo de cualquier idioma, evitando que las tildes, las "ñ" o los emojis se vean como caracteres extraños (ej: ``).

HTML

```html
<meta charset="UTF-8">
```

2.2.2.2 Viewport (para diseño responsive)

Esta es una de las etiquetas más importantes para el desarrollo web moderno. Le indica al navegador cómo controlar las dimensiones y la escala de la página en diferentes dispositivos.

- `width=device-width` : Iguala el ancho de la página al ancho del dispositivo.
- `initial-scale=1.0` : Establece el nivel de zoom inicial en 100%, sin alejar ni acercar el contenido. Sin esta línea, los sitios web se verían muy pequeños en los móviles, obligando al usuario a hacer zoom manualmente.

HTML

```html
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

2.2.2.3 Descripción (`description`)

Proporciona un resumen breve y conciso del contenido de la página. Los motores de búsqueda suelen mostrar esta descripción debajo del título en los resultados de búsqueda. Es tu oportunidad de "vender" tu contenido al usuario. Una buena descripción debe rondar los 150-160 caracteres.

HTML

```html
<meta name="description" content="Una descripción clara y concisa de tu página que invite al usuario a hacer clic.">
```

2.2.2.4 Palabras clave (`keywords`)

Históricamente, esta etiqueta se usaba para listar las palabras clave relevantes para la página. **Hoy en día, los principales motores de búsqueda como Google la ignoran por completo** debido al abuso y la manipulación (práctica conocida como *keyword stuffing*). Se considera una etiqueta obsoleta y no es necesario incluirla.

HTML

```html
<meta name="keywords" content="html, desarrollo web, programación">
```

2.2.2.5 Autor (`author`)

Identifica al autor del contenido de la página. Puede ser útil para blogs o artículos.

HTML

```html
<meta name="author" content="Tu Nombre o el Nombre de tu Empresa">
```

2.2.2.6 Robots

Da instrucciones específicas a los rastreadores (robots) de los motores de búsqueda.

- `index, follow`: Permite al robot indexar la página (mostrarla en los resultados) y seguir los enlaces que contiene. Es el comportamiento por defecto, por lo que no es estrictamente necesario añadirlo.

- `noindex, follow`: Impide que la página sea indexada, pero permite al robot seguir sus enlaces para descubrir otras páginas. Útil para páginas de agradecimiento, resultados de búsqueda internos o páginas de acceso.
- `index, nofollow`: Permite indexar la página, pero no seguir sus enlaces.
- `noindex, nofollow`: Impide tanto la indexación como el seguimiento de enlaces.

HTML

```html
<meta name="robots" content="index, follow">
```

2.2.3 Las Etiquetas `<link>`

La etiqueta `<link>` se usa para establecer relaciones entre el documento actual y recursos externos.

2.2.3.1 Hojas de estilo CSS

El uso más común es enlazar una hoja de estilos externa. El atributo `rel="stylesheet"` le dice al navegador que este archivo es una hoja de estilos que debe aplicar al documento.

HTML

```html
<link rel="stylesheet" href="estilos.css">
```

2.2.3.2 Favicon

Define el icono que se muestra en la pestaña del navegador, en los marcadores y en otras áreas de la interfaz. Aunque el formato `.ico` es el tradicional, también se usa `.png` o `.svg` y se pueden especificar diferentes tamaños para distintos dispositivos.

HTML

```html
<link rel="icon" href="favicon.ico" type="image/x-icon">
<link rel="icon" href="/images/favicon.png" type="image/png">
```

2.2.3.3 Fuentes web

Permite enlazar fuentes de servicios externos como Google Fonts. A menudo verás la etiqueta `<link rel="preconnect">` junto a la de la fuente. Esta etiqueta le indica al navegador que debe establecer una conexión con el dominio de las fuentes lo antes posible, lo que ayuda a que las fuentes se carguen más rápido y mejora el rendimiento de la página.

HTML

```html
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">
```

2.3 Metadatos para Redes Sociales: Open Graph y Twitter Cards

Cuando compartes un enlace en plataformas como Facebook, Twitter, WhatsApp o LinkedIn, estas no adivinan qué imagen o texto mostrar. En su lugar, buscan metadatos específicos en la página. El protocolo más extendido es **Open Graph (OG)**, usado por Facebook y muchos otros, mientras que Twitter tiene sus propias etiquetas (`twitter:card`), aunque también puede usar las de Open Graph si las suyas no están presentes.

Incluir estas etiquetas es crucial para controlar cómo se ve tu contenido al ser compartido.

- **`og:title`** : El título que se mostrará en la publicación.
- **`og:description`** : La descripción que aparecerá debajo del título.
- **`og:image`** : La URL de la imagen que quieres que se muestre. ¡Esta es la más importante para captar la atención!
- **`og:url`** : La URL canónica (la original) de la página.
- **`og:type`** : El tipo de contenido (ej: `website` , `article`).
- **`twitter:card`** : El tipo de "tarjeta" que Twitter debe mostrar (ej: `summary` , `summary_large_image`).

HTML

```html
<head>
  <title>10 Consejos para Aprender HTML</title>
  <meta name="description" content="Descubre los 10 mejores consejos para empezar a aprender HTML de forma rápida y sencilla.">

  <meta property="og:title" content="10 Consejos para Aprender HTML">
  <meta property="og:description" content="Descubre los 10 mejores consejos para empezar a aprender HTML de forma rápida y sencilla.">
  <meta property="og:image" content="https://www.tusitio.com/images/post-html.jpg">
  <meta property="og:url" content="https://www.tusitio.com/blog/10-consejos-html">
  <meta property="og:type" content="article">

  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="10 Consejos para Aprender HTML">
  <meta name="twitter:description" content="Descubre los 10 mejores consejos para empezar a aprender HTML de forma rápida y sencilla.">
  <meta name="twitter:image" content="https://www.tusitio.com/images/post-html.jpg">
</head>
```

2.4 Mejores Prácticas

1. **Siempre incluye estos meta tags básicos**: Son el esqueleto de una buena configuración inicial.

HTML

```html
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Título Descriptivo de la Página</title>
<meta name="description" content="Una descripción única y atractiva de esta página.">
```

2. **Mantén las descripciones únicas para cada página**: Si varias páginas tienen la misma descripción, los buscadores pueden interpretarlo como contenido duplicado y penalizar tu posicionamiento.

3. **Usa palabras clave de manera natural**: Aunque la etiqueta `<meta name="keywords">` esté obsoleta, las palabras clave siguen siendo importantes. Intégralas de forma lógica y natural en tu `<title>` y `<meta name="description">` pensando siempre en el usuario, no en el robot.

4. **Actualiza los metadatos cuando cambie el contenido**: Si modificas sustancialmente una página, asegúrate de que el título y la descripción sigan reflejando el nuevo contenido.

5.

Verifica que los enlaces a recursos externos sean correctos: Un error en la ruta (`href`) de tu CSS puede hacer que la página se vea completamente rota. Revisa siempre las rutas a tus estilos, favicons y fuentes.

2.5 Ejercicios

2.5.1 Ejercicio 1: `head` completo

Crea la sección `<head>` para una página de blog. Debe incluir:

- Codificación de caracteres **UTF-8**.
- **Viewport** para diseño adaptable.
- **Título**: "10 Consejos para Aprender HTML".
- **Descripción**: "Descubre los 10 mejores consejos para empezar a aprender HTML de forma rápida y sencilla".
- **Autor**: "Ana Pérez".
- Un enlace a la hoja de estilos `blog.css` ubicada en una carpeta `css`.

2.5.2 Ejercicio 2: Instrucciones para Robots

Escribe la etiqueta `<meta>` para indicar a los buscadores que **no indexen** una página pero que **sí sigan** los enlaces que contiene.

2.5.3 Ejercicio 3: Enlazar Recursos Externos

Escribe las etiquetas `<link>` necesarias para:

1. Añadir un **favicon** ubicado en `/images/logo.png` (tipo `image/png`).
2. Importar la fuente **"Lato"** desde Google Fonts.

2.5.4 Ejercicio 4: Metadatos para Redes Sociales

Añade las etiquetas de **Open Graph** necesarias para una página de producto cuyo título es "Zapatillas Deportivas X-Run" y cuya imagen principal está en `https://tienda.com/images/zapatillas-xrun.png`.

2.6 Soluciones

2.6.1 Solución Ejercicio 1

HTML

```html
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>10 Consejos para Aprender HTML</title>
  <meta name="description" content="Descubre los 10 mejores consejos para empezar a aprender HTML de forma rápida y sencilla.">
  <meta name="author" content="Ana Pérez">
  <link rel="stylesheet" href="css/blog.css">
</head>
```

2.6.2 Solución Ejercicio 2

HTML

```html
<meta name="robots" content="noindex, follow">
```

2.6.3 Solución Ejercicio 3

HTML

```html
<link rel="icon" href="/images/logo.png" type="image/png">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Lato:wght@400;700&display=swap" rel="stylesheet">
```

2.6.4 Solución Ejercicio 4

HTML

```html
<meta property="og:title" content="Zapatillas Deportivas X-Run">
<meta property="og:image" content="https://tienda.com/images/zapatillas-xrun.png">
<meta property="og:type" content="website">
```

2.7 Referencias y Recursos

2.7.1 Documentación Oficial

- MDN Web Docs - Metadatos HTML
- W3Schools - HTML Meta
- HTML.com - Meta Tags

2.7.2 SEO y Metadatos

- Google Search Central - Meta Tags
- Moz - Meta Description
- Schema.org
- The Open Graph protocol
- Twitter Cards Documentation

2.7.3 Herramientas

- Meta Tags Generator & Preview
- SEO Meta in 1 Click (Extensión de Chrome)
- Facebook Sharing Debugger

2.7.4 Tutoriales y Guías

- Codecademy - HTML Metadata
- CSS-Tricks - Essential Meta Tags for Social Media

3 Elementos de Texto en HTML

El texto es el pilar de cualquier página web. HTML nos ofrece un amplio conjunto de elementos para estructurar, dar significado (semántica) y formatear nuestro contenido textual. Esta guía explora los elementos esenciales, desde la estructura básica hasta el formato detallado.

3.1 Estructura Fundamental del Texto

Antes de dar formato, debemos estructurar el documento. Estos elementos definen la jerarquía y las divisiones básicas del contenido.

3.1.1 Encabezados (`<h1>` - `<h6>`)

Los encabezados son cruciales para organizar el contenido y para la accesibilidad y el SEO. Definen una jerarquía de títulos y subtítulos.

- `<h1>`: Título principal de la página. Debe haber solo **uno por página**.
- `<h2>`: Títulos de secciones principales.
- `<h3>` a `<h6>`: Subtítulos de niveles inferiores.

HTML

```html
<h1>El Título Principal de la Página</h1>
<p>Este es un párrafo introductorio.</p>
<h2>Primera Sección Principal</h2>
<p>Contenido de la primera sección.</p>
<h3>Un Subtítulo</h3>
<p>Más detalles sobre la sección.</p>
<h2>Segunda Sección Principal</h2>
<p>Contenido de la segunda sección.</p>
```

3.1.2 Párrafos (`<p>`)

El elemento `<p>` es el bloque de construcción fundamental para el texto. Agrupa frases y crea divisiones lógicas en el contenido.

HTML

```
<p>Este es un párrafo de texto. Los navegadores agregan automáticamente un espacio vertical
antes y después de cada párrafo para separarlos visualmente.</p>
```

3.1.3 Saltos de Línea (`<br>`)

El elemento `<br>` fuerza un salto de línea. Es útil para contenido donde el salto de línea es semánticamente importante, como en poemas o direcciones postales. No debe usarse para crear espacio entre elementos; para eso está CSS.

HTML

```
<address>
  Calle Falsa 123<br>
  Piso 4, Puerta B<br>
  28001, Madrid
</address>
```

3.1.4 Separadores Temáticos (`<hr>`)

El elemento `<hr>` representa una ruptura temática entre párrafos o secciones, como un cambio de escena en una historia. Visualmente, se muestra como una línea horizontal.

HTML

```
<p>...final del primer tema.</p>
<hr>
<p>Comienzo de un nuevo tema...</p>
```

3.2 Semántica y Formato de Texto en Línea

Estos elementos se usan dentro de bloques de texto (como un párrafo) para dar un significado o estilo específico a palabras o frases.

3.2.1 Énfasis y Fuerza (Semántico)

- `<strong>`: Indica que el texto tiene una **gran importancia**, seriedad o urgencia. Los navegadores lo muestran en **negrita**.
- `<em>`: Marca un texto con **énfasis**, como el que se usaría para acentuar una palabra en una conversación. Los navegadores lo muestran en *cursiva*.

HTML

```html
<p><strong>Advertencia:</strong> No tocar el cableado.</p>
<p>Debes hacerlo <em>ahora</em> mismo.</p>
```

3.2.2 Formato Tradicional (Presentacional)

Aunque es preferible usar `<strong>` y `<em>` por su valor semántico, estos elementos siguen siendo válidos para fines estilísticos donde no se implica un cambio de importancia o énfasis.

- `<b>`: Texto en negrita sin importancia semántica adicional (ej: palabras clave).
- `<i>`: Texto en cursiva para indicar un matiz diferente, como un término técnico, un pensamiento o el nombre de un barco.
- `<u>`: Texto subrayado. Su uso no se recomienda porque los usuarios lo confunden con un enlace.

HTML

```html
<p>La palabra clave es <b>renderizar</b>.</p>
<p>El barco <i>Titanic</i> se hundió en 1912.</p>
```

3.2.3 Subíndices, Superíndices y Resaltado

- `<sub>` y `<sup>`: Para subíndices y superíndices, comunes en fórmulas y notas al pie.
- `<mark>`: Para resaltar texto que es relevante en un contexto particular (como los resultados de una búsqueda).

HTML

```html
<p>Fórmula química: H<sub>2</sub>O</p>
<p>Teorema de Pitágoras: a<sup>2</sup> + b<sup>2</sup> = c<sup>2</sup></p>
<p>El buscador <mark>resaltó</mark> estos términos.</p>
```

3.2.4 Texto Editado

- `<del>`: Representa texto que ha sido eliminado.
- `<ins>`: Representa texto que ha sido insertado.

HTML

```html
<p>Precio anterior: <del>100€</del>. ¡Ahora <ins>75€</ins>!</p>
```

3.2.5 Contenedor Genérico en Línea (`<span>`)

El elemento `<span>` es un contenedor en línea que no tiene ningún significado semántico. Su principal uso es agrupar texto para aplicarle estilos con CSS o manipularlo con JavaScript, sin añadir saltos de línea ni otros cambios visuales por defecto.

HTML

```html
<p>Este es un texto normal con <span style="color: blue;">texto en azul</span> en medio.</p>
```

3.3 Citas, Referencias y Definiciones

Estos elementos dan contexto y atribución al contenido.

- `<blockquote>`: Para citas largas que ocupan su propio bloque.
- `<q>`: Para citas cortas que van dentro de una línea de texto.
- `<cite>`: Para citar el título de una obra creativa (libro, película, canción).
- `<address>`: Para información de contacto (autor, empresa).
- `<abbr>`: Para abreviaturas, mostrando el texto completo al pasar el ratón por encima.

HTML

```html
<blockquote cite="https://fuente.com">
  <p>Esta es una cita larga que se muestra con sangría.</p>
</blockquote>

<p>Como dijo el autor: <q>Las citas cortas van aquí</q>.</p>
<p>Mi libro favorito es <cite>Don Quijote de la Mancha</cite>.</p>

<address>
    Escrito por: Juan Pérez<br>
    Email: juan@ejemplo.com
</address>

<p>El <abbr title="World Wide Web Consortium">W3C</abbr> define los estándares web.</p>
```

3.4 Código y Texto Preformateado

Elementos para mostrar código o texto que respeta los espacios en blanco.

- `<code>`: Se usa para marcar una pieza de código fuente en línea.
- `<pre>`: Mantiene todos los espacios en blanco y saltos de línea del texto que contiene. Es ideal para mostrar bloques de código.

HTML

```html
<p>La función <code>console.log()</code> imprime en la consola.</p>

<pre>
  <code>
    function saludar() {
        console.log("¡Hola mundo!");
    }
  </code>
</pre>
```

3.5 Entidades HTML (Caracteres Especiales)

Algunos caracteres están reservados en HTML (como `<` y `>`). Si intentas usarlos en tu texto, el navegador podría confundirse y pensar que son etiquetas. Para mostrarlos correctamente, usamos **Entidades HTML**.

Las más comunes son:

CARÁCTER	ENTIDAD	DESCRIPCIÓN
`<`	`<`	*Less Than* (Menor que)
`>`	`>`	*Greater Than* (Mayor que)
`&`	`&`	*Ampersand* (Y comercial)
`"`	`"`	Comillas dobles
Espacio	` `	*Non-Breaking Space* (Espacio que no se rompe)
©	`©`	Copyright
€	`€`	Euro

Ejemplo:

HTML

```html
<p>Para escribir una etiqueta en negrita usa &lt;strong&gt;.</p>
```

3.6 Mejores Prácticas

1. **Uso Semántico**: Usa siempre el elemento que mejor describa el *significado* de tu contenido, no su apariencia. `<strong>` es para importancia, no solo para poner negrita.
2. **La Apariencia es de CSS**: Toda la estilización (colores, fuentes, márgenes, etc.) debe realizarse con CSS, no abusando de elementos HTML como `<br>` o `<b>`.
3. **Jerarquía de Encabezados**: No te saltes niveles de encabezados (ej: de un `<h2>` a un `<h4>`). Mantenlos en orden para una estructura lógica.
4. **Accesibilidad**: Usar los elementos correctos ayuda a los lectores de pantalla y a otras tecnologías de asistencia a interpretar correctamente tu página.

3.7 Ejercicios

3.7.1 Ejercicio 1: Estructura y Encabezados

Crea la estructura básica de una página de recetas. Debe tener un título principal (`<h1>`) para "Tarta de Manzana Casera", una sección (`<h2>`) para "Ingredientes" y otra (`<h2>`) para "Instrucciones". Dentro de "Instrucciones", añade un subtítulo (`<h3>`) para "Paso 1: Preparar la masa".

3.7.2 Ejercicio 2: Formato Básico

Crea un párrafo (`<p>`) que contenga la siguiente frase: "Este es un aviso muy importante. Presta atención a los detalles.". La palabra "importante" debe tener un **énfasis fuerte** (`<strong>`), y la palabra "atención" debe tener un **énfasis normal** (`<em>`). Adicionalmente, la palabra "detalles" debe estar dentro de un `<span>` con un estilo que le ponga el color rojo.

3.7.3 Ejercicio 3: Citas y Referencias

Escribe el código HTML para mostrar la siguiente cita de Albert Einstein en un bloque, atribuyendo la fuente a `https://www.brainyquote.com`: "La imaginación es más importante que el conocimiento.". Luego, en un párrafo separado, menciona que tu película favorita es `<cite>Interstellar</cite>` y añade la frase de un personaje usando una cita en línea: <q>No es posible, es necesario</q>.

3.7.4 Ejercicio 4: Elementos Especiales

Escribe el HTML necesario para mostrar correctamente la fórmula química del ácido sulfúrico (H_2SO_4) y el teorema de Pitágoras ($a^2 + b^2 = c^2$). En un nuevo párrafo, usa una abreviatura para "HyperText Markup Language" para que al pasar el ratón sobre **HTML** aparezca su significado completo.

3.7.5 Ejercicio 5: Código Preformateado

Muestra el siguiente bloque de código JavaScript en una página web, asegurándote de que los espacios y saltos de línea se mantengan exactamente como están escritos.

JAVASCRIPT

```javascript
function calcularTotal(precio, impuesto) {
  const total = precio * (1 + impuesto);
  return total;
}
```

3.8 Soluciones

3.8.1 Solución Ejercicio 1

HTML

```html
<h1>Tarta de Manzana Casera</h1>

<h2>Ingredientes</h2>
<h2>Instrucciones</h2>
<h3>Paso 1: Preparar la masa</h3>
```

3.8.2 Solución Ejercicio 2

HTML

```html
<p>Este es un aviso muy <strong>importante</strong>. Presta <em>atención</em> a los <span
style="color: red;">detalles</span>.</p>
```

3.8.3 Solución Ejercicio 3

HTML

```html
<blockquote cite="https://www.brainyquote.com">
  <p>La imaginación es más importante que el conocimiento.</p>
</blockquote>

<p>Mi película favorita es <cite>Interstellar</cite>, donde un personaje dice: <q>No es
posible, es necesario</q>.</p>
```

3.8.4 Solución Ejercicio 4

HTML

```html
<p>La fórmula del ácido sulfúrico es: H<sub>2</sub>SO<sub>4</sub>.</p>
<p>El teorema de Pitágoras es: a<sup>2</sup> + b<sup>2</sup> = c<sup>2</sup>.</p>
<p>El lenguaje <abbr title="HyperText Markup Language">HTML</abbr> es la base de la web.</p>
```

3.8.5 Solución Ejercicio 5

HTML

```html
<pre>
  <code>
    function calcularTotal(precio, impuesto) {
      const total = precio * (1 + impuesto);
      return total;
    }
</code>
</pre>
```

3.9 Recursos

- MDN - Elementos de Texto
- W3Schools - HTML Text Formatting
- HTML Living Standard - Text-level semantics

4 Listas en HTML

Las listas son elementos fundamentales en HTML que permiten organizar y estructurar contenido en forma de ítems. HTML ofrece tres tipos principales de listas: ordenadas, no ordenadas y de definición.

4.1 Listas Ordenadas (ol)

Las listas ordenadas muestran elementos en una secuencia numerada. Son ideales para pasos, rankings o cualquier contenido que requiera un orden específico.

HTML

```html
<ol>
  <li>Primer elemento</li>
  <li>Segundo elemento</li>
  <li>Tercer elemento</li>
</ol>
```

1. Primer elemento
2. Segundo elemento
3. Tercer elemento

4.1.1 Atributos de las listas ordenadas

4.1.1.1 type

Define el tipo de numeración:

HTML

```html
<ol type="1"> <!-- Números (por defecto) -->
<ol type="A"> <!-- Letras mayúsculas -->
```

```html
<ol type="a"> <!-- Letras minúsculas -->
<ol type="I"> <!-- Números romanos mayúsculas -->
<ol type="i"> <!-- Números romanos minúsculas -->
```

1. Primer elemento 2. Segundo elemento 3. Tercer elemento	A. Primer elemento B. Segundo elemento C. Tercer elemento	a. Primer elemento b. Segundo elemento c. Tercer elemento	I. Primer elemento II. Segundo elemento III. Tercer elemento	i. Primer elemento ii. Segundo elemento iii. Tercer elemento

4.1.1.2 start

Define el número inicial de la lista:

HTML

```html
<ol start="5">
  <li>Comenzará en 5</li>
  <li>Continuará en 6</li>
</ol>
```

5. Comenzará en 5
6. Continuará en 6

4.1.1.3 reversed

Invierte el orden de la numeración:

HTML

```html
<ol reversed>
  <li>Tercer elemento</li>
  <li>Segundo elemento</li>
  <li>Primer elemento</li>
</ol>
```

3. Tercer elemento
2. Segundo elemento
1. Primer elemento

4.2 Listas No Ordenadas (ul)

Las listas no ordenadas se utilizan cuando el orden de los elementos no es relevante. Por defecto, los elementos se marcan con viñetas.

HTML

```html
<ul>
  <li>Elemento</li>
  <li>Otro elemento</li>
  <li>Un elemento más</li>
</ul>
```

- Elemento
- Otro elemento
- Un elemento más

4.2.1 Estilos de viñetas

Se pueden modificar mediante CSS:

HTML

```html
<ul style="list-style-type: disc">    <!-- Círculo relleno (por defecto) -->
<ul style="list-style-type: circle">  <!-- Círculo vacío -->
<ul style="list-style-type: square">  <!-- Cuadrado -->
<ul style="list-style-type: none">    <!-- Sin viñeta -->
```

- Elemento
- Otro elemento
- Un elemento más

- Elemento
- Otro elemento
- Un elemento más

- Elemento
- Otro elemento
- Un elemento más

Elemento
Otro elemento
Un elemento más

4.3 Listas de Definición (dl)

Las listas de definición se utilizan para crear un glosario o lista de términos con sus definiciones.

HTML

```html
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language, el lenguaje estándar para crear páginas web.</dd>

  <dt>CSS</dt>
  <dd>Cascading Style Sheets, el lenguaje usado para estilizar páginas web.</dd>
</dl>
```

- `<dt>`: Define el término (Definition Term)
- `<dd>`: Define la descripción (Definition Description)

4.4 Listas Anidadas

Las listas pueden contener otras listas, creando una estructura jerárquica.

HTML

```html
<ul>
  <li>Frutas
    <ul>
      <li>Cítricos
        <ul>
          <li>Naranja</li>
          <li>Limón</li>
        </ul>
      </li>
      <li>Tropicales
        <ul>
          <li>Mango</li>
          <li>Piña</li>
        </ul>
      </li>
    </ul>
```

```
  </li>
</ul>
```

- Frutas
  - Cítricos
    - Naranja
    - Limón
  - Tropicales
    - Mango
    - Piña

4.4.1 Buenas Prácticas para Listas Anidadas

1. **Indentación**: Mantén una indentación clara para cada nivel
2. **Profundidad**: Evita anidar más de 3 niveles para mantener la legibilidad
3. **Consistencia**: Usa el mismo tipo de lista para elementos similares
4. **Semántica**: Elige el tipo de lista según el significado del contenido

4.5 Casos de Uso Comunes

4.5.1 Menús de Navegación

HTML

```html
<nav>
  <ul>
    <li><a href="#inicio">Inicio</a></li>
    <li><a href="#servicios">Servicios</a></li>
    <li><a href="#contacto">Contacto</a></li>
  </ul>
</nav>
```

4.5.2 Pasos de un Proceso

HTML

```html
<ol>
  <li>Registrarse en la plataforma</li>
  <li>Verificar el correo electrónico</li>
  <li>Completar el perfil</li>
  <li>Comenzar a usar el servicio</li>
</ol>
```

4.5.3 Glosario de Términos

HTML

```html
<dl>
  <dt>FAQ</dt>
  <dd>Frequently Asked Questions (Preguntas Frecuentes)</dd>

  <dt>CMS</dt>
  <dd>Content Management System (Sistema de Gestión de Contenidos)</dd>
</dl>
```

4.6 Ejercicios

4.6.1 Ejercicio 1: Lista Ordenada con Atributos

Crea una lista con un ranking de los 3 mejores postres. La lista debe ser **ordenada** (`<ol>`) y mostrarse en **orden inverso** (`reversed`). Además, la numeración debe usar **números romanos en mayúsculas** (`type="I"`).

Los elementos son:

- Tarta de queso
- Tiramisú
- Brownie con helado

4.6.2 Ejercicio 2: Listas Anidadas

Crea una lista de compras **no ordenada** (`<ul>`) dividida por categorías. La lista principal debe contener las categorías "Lácteos" y "Frutas". Dentro de cada categoría, anida otra lista no ordenada que use **viñetas cuadradas** (`square`) y contenga al menos dos productos.

4.6.3 Ejercicio 3: Lista de Definición

Elabora un pequeño glosario utilizando una **lista de definición** (`<dl>`). Define los siguientes dos términos:

1. **Término**: "DOM". **Definición**: "Document Object Model, una interfaz de programación para documentos HTML".
2. **Término**: "URL". **Definición**: "Uniform Resource Locator, la dirección de un recurso en la World Wide Web".

4.6.4 Ejercicio 4: Menú de Navegación

Construye una estructura básica para un **menú de navegación** de un sitio web. Utiliza una lista no ordenada dentro de una etiqueta `<nav>`. Los elementos de la lista deben ser enlaces (etiqueta `<a>`) con un `href` temporal (`#`) para "Inicio", "Sobre nosotros" y "Contacto".

4.7 Soluciones

4.7.1 Solución Ejercicio 1

HTML

```html
<ol type="I" reversed>
  <li>Tarta de queso</li>
  <li>Tiramisú</li>
  <li>Brownie con helado</li>
</ol>
```

4.7.2 Solución Ejercicio 2

HTML

```html
<ul>
  <li>Lácteos
    <ul style="list-style-type: square;">
      <li>Leche</li>
      <li>Yogur</li>
    </ul>
  </li>
  <li>Frutas
    <ul style="list-style-type: square;">
      <li>Manzanas</li>
      <li>Plátanos</li>
    </ul>
  </li>
</ul>
```

4.7.3 Solución Ejercicio 3

HTML

```html
<dl>
  <dt>DOM</dt>
  <dd>Document Object Model, una interfaz de programación para documentos HTML.</dd>
  <dt>URL</dt>
  <dd>Uniform Resource Locator, la dirección de un recurso en la World Wide Web.</dd>
</dl>
```

4.7.4 Solución Ejercicio 4

HTML

```html
<nav>
  <ul>
    <li><a href="#">Inicio</a></li>
    <li><a href="#">Sobre nosotros</a></li>
    <li><a href="#">Contacto</a></li>
  </ul>
</nav>
```

4.8 Referencias y Recursos

4.8.1 Documentación Oficial

- MDN Web Docs - Listas HTML
- W3Schools - HTML Lists
- HTML.com - Lists

4.8.2 Tutoriales y Guías

- Codecademy - HTML Lists
- FreeCodeCamp - HTML Lists Guide
- CSS-Tricks - List Styling

5 Enlaces en HTML

Los enlaces (o hipervínculos) son elementos fundamentales en HTML que permiten conectar páginas web y recursos entre sí. Son la base de la navegación en internet.

5.1 Sintaxis Básica

El elemento `<a>` (anchor) se usa para crear enlaces:

HTML

```html
<a href="destino.html">Texto del enlace</a>
```

Donde:

- `href` es el atributo que especifica el destino
- El contenido entre las etiquetas es lo que verá el usuario

5.2 Tipos de Enlaces

5.2.1 Enlaces Externos

Apuntan a otros sitios web:

HTML

```html
<a href="https://www.ejemplo.com">Visita Ejemplo.com</a>
```

5.2.2 Enlaces Internos

Apuntan a páginas dentro del mismo sitio:

HTML

```html
<a href="/contacto.html">Contacto</a>
<a href="./productos/catalogo.html">Ver Catálogo</a>
```

5.2.3 Enlaces a Correo Electrónico

Abren el cliente de correo predeterminado:

HTML

```html
<a href="mailto:contacto@ejemplo.com">Envíanos un email</a>
```

5.2.4 Enlaces a Teléfono

Útiles especialmente en dispositivos móviles:

HTML

```html
<a href="tel:+34600000000">Llámanos</a>
```

5.3 Rutas de Enlaces

5.3.1 Rutas Absolutas

- Incluyen el protocolo y dominio completo
- Se usan principalmente para enlaces externos

HTML

```
<a href="https://www.ejemplo.com/pagina.html">Enlace absoluto</a>
```

5.3.2 Rutas Relativas (Navegando entre carpetas)

Las rutas relativas dependen de **dónde está el archivo que estás editando ahora mismo**. Son las más usadas porque si mueves tu carpeta de proyecto, los enlaces siguen funcionando.

Imagina esta estructura:

- `index.html`
- `contacto.html`
- `imagenes/`
  - `foto.jpg`
- `blog/`
  - `articulo.html`

Cómo moverte:

1. **Mismo nivel (`./` o nada):** Si estás en `index.html` y quieres ir a `contacto.html`:

HTML

```html
<a href="contacto.html">Ir a Contacto</a>
```

2. **Entrar en una carpeta (`nombre_carpeta/`):** Si estás en `index.html` y quieres cargar la foto:

HTML

```html
<img src="imagenes/foto.jpg" alt="Foto">
```

3. **Salir de una carpeta (`../`):** Si estás dentro de `blog/articulo.html` y quieres volver al `index.html` (que está "arriba"):

HTML

```html
<a href="../index.html">Volver al Inicio</a>
```

> ***Truco:*** *Cada `../` te sube un nivel. Si necesitas subir dos niveles, usa `../../`.*

5.4 Atributos de Enlaces

5.4.1 target

Define dónde se abrirá el enlace:

HTML

```html
<!-- Nueva pestaña -->
<a href="https://ejemplo.com" target="_blank">Abrir en nueva pestaña</a>

<!-- Misma ventana (por defecto) -->
<a href="pagina.html" target="_self">Abrir aquí</a>
```

5.4.2 rel

Define la relación con la página enlazada:

HTML

```html
<a href="https://ejemplo.com" rel="nofollow">Sin seguimiento</a>
<a href="documento.pdf" rel="noopener noreferrer" target="_blank">Documento seguro</a>
```

5.4.3 download

Sugiere descargar el recurso en lugar de navegarlo:

HTML

```html
<a href="archivo.pdf" download>Descargar PDF</a>
<a href="imagen.jpg" download="foto.jpg">Descargar imagen</a>
```

5.5 Enlaces Internos en la Página

5.5.1 Anclas con ID

Permiten saltar a secciones específicas de la página:

HTML

```html
<!-- Destino -->
<h2 id="seccion">Sección</h2>

<!-- Enlaces al destino -->
<a href="#seccion">Ir a sección</a>
```

5.5.2 Enlaces de Navegación

Común en menús y tablas de contenido:

HTML

```html
<nav>
  <ul>
    <li><a href="#inicio">Inicio</a></li>
    <li><a href="#servicios">Servicios</a></li>
    <li><a href="#contacto">Contacto</a></li>
  </ul>
</nav>
```

5.6 Mejores Prácticas

1. **Texto descriptivo**

HTML

```html
<!-- Bien -->
<a href="productos.html">Ver catálogo de productos</a>

<!-- Evitar -->
<a href="productos.html">Click aquí</a>
```

1. **Enlaces externos seguros**

HTML

```html
<a href="https://externo.com"
   target="_blank"
   rel="noopener noreferrer">
   Sitio externo
</a>
```

1. **Indicar enlaces externos**

HTML

```html
<a href="https://ejemplo.com">
  Visitar ejemplo.com
  <span class="external-icon">↗</span>
</a>
```

1. **Enlaces accesibles**

HTML

```html
<a href="documento.pdf"
   aria-label="Descargar informe anual (PDF, 2MB)">
   Informe anual
</a>
```

5.7 Ejercicios

5.7.1 Ejercicio 1: Tipos de Enlaces

Crea los siguientes tres enlaces:

1. Un enlace **externo** que apunte a `https://www.wikipedia.org` y se abra en una **nueva pestaña**. El texto del enlace debe ser "Visita la Wikipedia".
2. Un enlace que inicie un borrador de **correo electrónico** dirigido a `soporte@ejemplo.com`.
3. Un enlace para iniciar una **llamada telefónica** al número `+34 900 123 456`.

5.7.2 Ejercicio 2: Rutas y Anclas

Crea una estructura de navegación interna.

1. Primero, define dos secciones en tu página con encabezados `<h2>`. Asígnales los `id` `resumen` y `detalles`, respectivamente.
2. Luego, en la parte superior de la página, crea una lista de enlaces que apunten a estas **anclas**.
3. Finalmente, añade un enlace que apunte a una imagen llamada `logo.png` que se encuentra en un **subdirectorio** llamado `img`.

5.7.3 Ejercicio 3: Atributos `download` y `rel`

Crea dos enlaces con atributos específicos:

1. Un enlace para **descargar** un archivo llamado `manual.pdf`. El nombre del archivo al guardar debe ser `guia-usuario.pdf`.
2. Un enlace **externo** a `https://sitio-asociado.com` que se abra en una nueva pestaña. Por seguridad, debe incluir los atributos `rel` con los valores `noopener` y `noreferrer`.

5.7.4 Ejercicio 4: Mejores Prácticas

Corrige el siguiente enlace para que siga las buenas prácticas de **accesibilidad y SEO**, reemplazando el texto genérico por uno descriptivo.

HTML

```html
<p>Para saber más, <a href="articulos/html-semantico.html">haz click aquí</a>.</p>
```

5.8 Soluciones

5.8.1 Solución Ejercicio 1

HTML

```html
<a href="https://www.wikipedia.org" target="_blank">Visita la Wikipedia</a>
<a href="mailto:soporte@ejemplo.com">Contactar por email</a>
<a href="tel:+34900123456">Llamar ahora</a>
```

5.8.2 Solución Ejercicio 2

HTML

```html
<ul>
  <li><a href="#resumen">Ir al resumen</a></li>
  <li><a href="#detalles">Ver detalles</a></li>
</ul>

<a href="images/logo.png">Ver logo</a>

<h2 id="resumen">Resumen del Contenido</h2>
<p>...</p>

<h2 id="detalles">Detalles Específicos</h2>
<p>...</p>
```

5.8.3 Solución Ejercicio 3

HTML

```html
<a href="manual.pdf" download="guia-usuario.pdf">Descargar la guía del usuario</a>

<a href="https://sitio-asociado.com" target="_blank" rel="noopener noreferrer">Visita nuestro
sitio asociado</a>
```

5.8.4 Solución Ejercicio 4

HTML

```html
<p>Aprende más en nuestro <a href="articulos/html-semantico.html">artículo sobre HTML
semántico</a>.</p>
```

5.9 Recursos

5.9.1 Documentación Oficial

- MDN Web Docs - Enlaces HTML
- W3Schools - HTML Links
- HTML.com - Hyperlinks

6 Tablas en HTML

Las tablas en HTML se utilizan para mostrar **datos tabulares** en un formato de filas y columnas. Son la herramienta semántica correcta para presentar información que tiene una relación lógica en dos dimensiones (filas y columnas), como informes financieros, horarios, estadísticas o listados de productos. Es una mala práctica utilizarlas para maquetar el diseño de una página web; para ese propósito, se deben usar CSS Flexbox o Grid.

6.1 Estructura Básica

Toda tabla se construye a partir de un conjunto de etiquetas que definen su estructura y contenido de manera jerárquica.

HTML

```html
<table>
  <caption>Título Descriptivo de la Tabla</caption>
  <thead>
    <tr>
      <th>Encabezado de Columna 1</th>
      <th>Encabezado de Columna 2</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Dato Fila 1, Columna 1</td>
      <td>Dato Fila 1, Columna 2</td>
    </tr>
    <tr>
      <td>Dato Fila 2, Columna 1</td>
      <td>Dato Fila 2, Columna 2</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Resumen Columna 1</td>
      <td>Resumen Columna 2</td>
    </tr>
  </tfoot>
</table>
```

Título Descriptivo de la Tabla

Encabezado de Columna 1	Encabezado de Columna 2
Dato Fila 1, Columna 1	Dato Fila 1, Columna 2
Dato Fila 2, Columna 1	Dato Fila 2, Columna 2
Resumen Columna 1	Resumen Columna 2

6.1.1 Elementos Principales

6.1.1.1 `<table>`

- Es el **elemento raíz** que envuelve todo el contenido de la tabla.
- Define el inicio y fin de la tabla. Todos los demás elementos de la tabla deben estar anidados dentro de él.
- Puede tener atributos como `border`, `cellspacing` y `cellpadding`, aunque **estos son atributos obsoletos**. La práctica moderna es dar estilo a las tablas utilizando CSS para una mayor flexibilidad y separación de responsabilidades (estructura vs. presentación).

`<tr>` 6.1.1.2 (Table Row)

- Define una **fila** dentro de la tabla.
- Actúa como un contenedor para las celdas de esa fila, que pueden ser celdas de encabezado (`<th>`) o celdas de datos (`<td>`).
- Se utiliza dentro de las secciones `<thead>`, `<tbody>` y `<tfoot>`.

`<td>` 6.1.1.3 (Table Data)

- Define una **celda de datos estándar**.
- Contiene el contenido principal de la tabla (los datos). El contenido puede ser texto, imágenes, enlaces o cualquier otro elemento HTML.
- Puede expandirse para ocupar el espacio de múltiples columnas o filas utilizando los atributos `colspan` y `rowspan`, respectivamente.

`<th>` 6.1.1.4 (Table Header)

- Define una **celda de encabezado**. Por defecto, los navegadores suelen mostrar su contenido en **negrita y centrado**.
- Aporta un valor semántico crucial: indica que esa celda es el título de una columna o de una fila.
- Mejora la **accesibilidad**, ya que los lectores de pantalla la utilizan para anunciar el contexto de cada celda de datos al usuario.

6.2 Secciones de Tabla

Agrupar las filas en secciones de encabezado, cuerpo y pie no solo organiza el código, sino que también aporta beneficios semánticos y funcionales.

6.2.1 `<thead>`

- Define la sección de **encabezado** de la tabla.
- Debe contener una o más filas (`<tr>`) con los títulos de las columnas (`<th>`).
- Aunque se coloque en el código, los navegadores siempre lo renderizarán en la parte superior.
- En tablas muy largas que se imprimen, el contenido de `<thead>` puede repetirse en la parte superior de cada página.

6.2.2 `<tbody>`

- Contiene el **cuerpo principal o el contenido de datos** de la tabla.
- Se coloca después de `<thead>` y antes de `<tfoot>` .
- Una tabla puede tener **múltiples elementos** `<tbody>` para agrupar filas de datos relacionadas, lo que permite aplicar estilos o scripts a secciones específicas.

6.2.3 `<tfoot>`

- Define la sección de **pie** de la tabla.
- Es útil para mostrar resúmenes, totales o notas al final de las columnas.
- Semánticamente, puede declararse en el código HTML antes o después del `<tbody>`, pero los navegadores siempre lo mostrarán **al final de la tabla**.

6.3 Spanning y Grupos

`colspan` 6.3.1 y `rowspan`

Estos atributos permiten que una celda (`<td>` o `<th>`) se extienda a través de múltiples columnas o filas, creando diseños de tabla más complejos.

- **`colspan="numero"`**: Hace que la celda se expanda horizontalmente para ocupar el espacio de `numero` columnas.
- **`rowspan="numero"`**: Hace que la celda se expanda verticalmente para ocupar el espacio de `numero` filas.

HTML

```html
<table>
  <tr>
    <th colspan="2">Esta celda de encabezado ocupa dos columnas</th>
  </tr>
  <tr>
    <td rowspan="2">Esta celda de datos ocupa dos filas</td>
    <td>Celda normal (Fila 1, Columna 2)</td>
  </tr>
  <tr>
    <td>Celda normal (Fila 2, Columna 2)</td>
  </tr>
</table>
```

Esta celda de encabezado ocupa dos columnas	
Esta celda de datos ocupa dos filas	Celda normal (Fila 1, Columna 2)
	Celda normal (Fila 2, Columna 2)

6.3.2 Grupos de Columnas: `<colgroup>` y `<col>`

Estos elementos permiten agrupar y aplicar estilos a una o más columnas sin tener que añadir clases a cada celda individualmente.

- **`<colgroup>`**: Es el contenedor para uno o más elementos `<col>`. Debe ser un hijo directo de `<table>`.
- **`<col>`**: Especifica las propiedades para una columna. Se puede usar el atributo `span` para que un solo `<col>` aplique su estilo a varias columnas consecutivas.

HTML

```html
<table>
  <colgroup>
    <col style="background-color: #f2f2f2">
    <col span="2" style="background-color: #e6e6e6">
  </colgroup>
  <tr>
    <th>ISBN</th>
    <th>Título</th>
    <th>Precio</th>
  </tr>
  <tr>
    <td>978-3-16-148410-0</td>
    <td>El Gran Libro</td>
    <td>25€</td>
  </tr>
</table>
```

ISBN	Título	Precio
978-3-16-148410-0	El Gran Libro	25€

6.4 Tablas Responsivas

Hacer que las tablas se vean bien en dispositivos móviles es un desafío común. Aquí hay dos enfoques principales.

6.4.1 Contenedor Responsivo (Scroll Horizontal)

La técnica más simple y robusta es envolver la tabla en un `div` y permitir el desbordamiento horizontal. Esto evita que la tabla "rompa" el layout de la página en pantallas pequeñas.

HTML

```html
<div class="table-responsive">
  <table>
    </table>
</div>
```

6.4.1.1 CSS Básico para Responsividad

Este CSS hace que el contenedor `div` muestre una barra de scroll horizontal solo cuando el ancho de la tabla excede el ancho del contenedor.

CSS

```css
.table-responsive {
  overflow-x: auto; /* Permite el scroll horizontal */
  max-width: 100%;  /* Asegura que el contenedor no se salga de la pantalla */
}
```

6.4.2 Técnicas Avanzadas (Reflow con Atributos `data-`)

Una técnica más avanzada consiste en reestructurar la tabla en un formato de lista en pantallas pequeñas usando CSS. Se añaden atributos `data-*` a las celdas para usarlos como etiquetas visuales con pseudo-elementos CSS.

HTML

```html
<table>
  <thead>
    <tr>
      <th>Producto</th>
      <th>Precio</th>
```

```
      <th>Stock</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td data-label="Producto:">Laptop Gamer</td>
      <td data-label="Precio:">1.299€</td>
      <td data-label="Stock:">15</td>
    </tr>
  </tbody>
</table>
```

En CSS, se usarían *Media Queries* para ocultar el `<thead>` en móviles y mostrar el contenido del atributo `data-label` antes del contenido de cada `<td>`.

6.5 Mejores Prácticas

1. **Usa tablas solo para datos tabulares**
   - No las uses para maquetar la página (posicionar elementos). Para eso existe CSS Flexbox y Grid.
   - Úsalas cuando la información tenga una relación clara entre filas y columnas.
   - Son ideales para comparar datos o mostrar conjuntos de información estructurada.
2. **Incluye siempre encabezados (`<th>`)**
   - Usa `<th>` para los títulos de filas y columnas. Esto es semánticamente correcto.
   - Combínalos con el atributo `scope` para indicar explícitamente si es un encabezado de columna (`scope="col"`) o de fila (`scope="row"`).
   - Esto mejora drásticamente la accesibilidad para usuarios de lectores de pantalla.
3. **Mantén un diseño consistente y legible**
   - Alinea los datos de manera lógica (ej. texto a la izquierda, números a la derecha).
   - Utiliza espaciado y bordes (con CSS) para que la tabla sea fácil de leer.
   - Asegúrate de que todas las filas del cuerpo de la tabla tengan el mismo número de columnas (o un `colspan` que lo justifique).

4. **Prioriza la accesibilidad**

- Usa la etiqueta `<caption>` para dar un título descriptivo a la tabla.
- Usa `<th>` con el atributo `scope` para definir la relación encabezado-dato.
- En tablas muy complejas, usa el atributo `headers` en las celdas `<td>` para asociarlas explícitamente a sus `<th>` correspondientes.
- Asegura un buen contraste de color entre el texto y el fondo.

6.6 Atributos Útiles para Accesibilidad

6.6.1 `<caption>`

Proporciona un título o descripción para la tabla. Debe ser el **primer elemento hijo** de la etiqueta `<table>`. Es leído por los lectores de pantalla para dar contexto al usuario antes de leer los datos.

HTML

```html
<table>
  <caption>Gastos Mensuales del Departamento de Marketing - Q3</caption>
  </table>
```

6.6.2 `scope`

Define el alcance de un encabezado (`<th>`). Aclara si el encabezado se aplica a toda la columna (`col`), a toda la fila (`row`), o a un grupo de columnas (`colgroup`) o filas (`rowgroup`).

HTML

```html
<thead>
  <tr>
    <th scope="col">Nombre del Producto</th>
    <th scope="col">Unidades Vendidas</th>
  </tr>
</thead>
<tbody>
  <tr>
    <th scope="row">Producto A</th> <td>1,200</td>
  </tr>
</tbody>
```

6.6.3 `headers`

Se usa en una celda de datos (`<td>`) para crear una asociación explícita con una o más celdas de encabezado (`<th>`). Su valor es una lista de los `id` de los encabezados correspondientes, separados por espacios. Es especialmente útil en tablas con estructuras irregulares o múltiples niveles de encabezados.

HTML

```html
<th id="nombre">Nombre</th>
<th id="ventas">Ventas</th>
<td headers="nombre ventas">Celda de dato</td>
```

6.7 Ejercicios

6.7.1 Ejercicio 1: Estructura Básica de una Tabla

Crea una tabla simple que muestre los nombres y edades de dos personas. La tabla debe tener un **título principal (caption)** que diga "Datos Personales". Utiliza `<thead>` para los encabezados ("Nombre" y "Edad") y `<tbody>` para los datos de las personas.

6.7.2 Ejercicio 2: Combinar Celdas (`colspan` y `rowspan`)

Diseña una tabla que represente un horario simple. La tabla debe tener una celda de encabezado para "Horario". La primera fila debe tener una celda que ocupe dos columnas (`colspan="2"`) con el texto "Mañana". La primera columna debe tener una celda que ocupe dos filas (`rowspan="2"`) con el texto "8:00 - 10:00".

6.7.3 Ejercicio 3: Secciones de Tabla y Accesibilidad

Crea una tabla de precios para tres productos. La tabla debe estar estructurada con `<thead>`, `<tbody>` y `<tfoot>`.

- En el `<thead>`, los encabezados "Producto" y "Precio" deben usar el atributo `scope="col"`.
- En el `<tfoot>`, la primera celda debe ocupar una columna y decir "Total", mientras que la segunda debe mostrar un precio total (ej: 350€).

6.7.4 Ejercicio 4: Tabla Compleja

Construye una tabla que muestre los resultados de una pequeña liga de tres equipos.

- **Encabezados**: Equipo, Partidos Jugados (PJ), Puntos.

- Usa `<thead>` y `<tbody>`.
- Añade una fila de encabezado adicional dentro del `<tbody>` que diga "Grupo A" y ocupe las tres columnas (`colspan="3"`).
- El primer equipo de la lista debe tener un `<th>` como encabezado de fila con el `scope="row"`.

6.8 Soluciones

6.8.1 Solución Ejercicio 1

HTML

```html
<table>
  <caption>Datos Personales</caption>
  <thead>
    <tr>
      <th>Nombre</th>
      <th>Edad</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ana</td>
      <td>28</td>
    </tr>
    <tr>
      <td>Luis</td>
      <td>32</td>
    </tr>
  </tbody>
</table>
```

6.8.2 Solución Ejercicio 2

HTML

```html
<table border="1">
  <caption>Horario Semanal</caption>
  <thead>
    <tr>
      <th colspan="2">Mañana</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td rowspan="2">8:00 - 10:00</td>
      <td>Matemáticas</td>
    </tr>
    <tr>
      <td>Historia</td>
    </tr>
  </tbody>
</table>
```

6.8.3 Solución Ejercicio 3

HTML

```html
<table>
  <caption>Lista de Precios</caption>
  <thead>
    <tr>
      <th scope="col">Producto</th>
      <th scope="col">Precio</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Producto A</td>
      <td>100€</td>
    </tr>
    <tr>
      <td>Producto B</td>
      <td>150€</td>
    </tr>
    <tr>
      <td>Producto C</td>
      <td>100€</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Total</td>
      <td>350€</td>
    </tr>
  </tfoot>
</table>
```

6.8.4 Solución Ejercicio 4

HTML

```html
<table border="1">
  <caption>Resultados de la Liga</caption>
  <thead>
    <tr>
      <th scope="col">Equipo</th>
      <th scope="col">Partidos Jugados (PJ)</th>
      <th scope="col">Puntos</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th colspan="3">Grupo A</th>
    </tr>
    <tr>
      <th scope="row">Equipo Alfa</th>
      <td>10</td>
      <td>25</td>
    </tr>
    <tr>
      <td>Equipo Beta</td>
```

```
      <td>10</td>
      <td>18</td>
    </tr>
    <tr>
      <td>Equipo Gamma</td>
      <td>10</td>
      <td>12</td>
    </tr>
  </tbody>
</table>
```

6.9 Referencias y Recursos

6.9.1 Documentación Oficial

- MDN Web Docs - Table
- W3Schools - HTML Tables
- HTML.com - Tables

6.9.2 Tutoriales y Guías

- CSS-Tricks - A Complete Guide to the Table Element
- FreeCodeCamp - HTML Tables

6.9.3 Herramientas

- Table Generator
- HTML Table To JSON Converter
- HTML Table Styler

7 Multimedia y Contenido Embebido

HTML5 proporciona un robusto conjunto de elementos para integrar contenido multimedia y embebido, como imágenes, audio, video y documentos externos, directamente en las páginas web. Gestionar estos elementos de manera eficiente es clave para el rendimiento, la accesibilidad y la experiencia de usuario.

7.1 Imágenes

Las imágenes son un componente fundamental del contenido web. HTML ofrece varias maneras de mostrarlas eficientemente.

7.1.1 Elemento `<img>`

Es el elemento principal para mostrar imágenes estáticas.

Sintaxis básica:

HTML

```html
<img src="ruta/imagen.jpg" alt="Descripción de la imagen" width="300" height="200">
```

Atributos clave:

- `src`: (Obligatorio) Especifica la ruta del archivo de imagen.
- `alt`: (Obligatorio) Proporciona un texto alternativo por si la imagen no se puede mostrar, esencial para la accesibilidad.
- `width` / `height`: Define las dimensiones de la imagen para que el navegador reserve el espacio y evite saltos de contenido (layout shifts).

- `loading`: Permite controlar la carga de la imagen. El valor `lazy` difiere la carga hasta que la imagen está cerca de aparecer en la pantalla, optimizando el rendimiento.

7.1.2 Elemento `<picture>`

Este elemento ofrece una mayor flexibilidad para el diseño responsivo (*art direction*), permitiendo al navegador elegir la imagen más adecuada según el tamaño de la pantalla u otras condiciones.

HTML

```html
<picture>
  <source media="(min-width: 800px)" srcset="imagen-grande.jpg">
  <source media="(min-width: 400px)" srcset="imagen-mediana.jpg">
  <img src="imagen-pequena.jpg" alt="Imagen responsive que se adapta al tamaño de pantalla">
</picture>
```

7.1.3 Formatos de Imagen Recomendados

- **JPG**: Ideal para fotografías y imágenes complejas con muchos colores.
- **PNG**: Perfecto para imágenes que requieren transparencia, como logos o iconos.
- **WebP**: Un formato moderno que ofrece una compresión superior (con y sin pérdida) y mejor calidad que JPG y PNG.
- **SVG**: Para gráficos vectoriales, como logos e iconos, que necesitan escalar a cualquier tamaño sin perder calidad.
- **GIF**: Usado principalmente para animaciones simples.

7.1.4 SVG en Línea (Inline SVG)

Además de usar SVG en una etiqueta `<img>`, una técnica muy potente es **incrustar el código SVG directamente en el documento HTML**. Esto ofrece un control total sobre la imagen, permitiendo su manipulación con CSS y JavaScript.

Sintaxis básica:

HTML

```html
<svg
  xmlns="http://www.w3.org/2000/svg"
  width="24"
  height="24"
  viewBox="0 0 24 24"
  fill="none"
  stroke="currentColor"
  stroke-width="2"
  stroke-linecap="round"
  stroke-linejoin="round"
  class="icono-check"
  aria-labelledby="tituloIconoCheck"
>
  <title id="tituloIconoCheck">Icono de confirmación</title>
  <path d="M20 6 9 17l-5-5"></path>
</svg>
```

Ventajas clave:

- **Estilo con CSS**: Puedes aplicar estilos directamente a los elementos del SVG. Por ejemplo, puedes cambiar el color (`fill`), el borde (`stroke`) o el tamaño del icono con una simple regla CSS.

  CSS

  ```css
  .icono-check {
    stroke: green;
    width: 48px; /* El SVG escala sin perder calidad */
  }
  .icono-check:hover {
    stroke: darkgreen;
    transform: scale(1.1);
  }
  ```

- **Menos peticiones HTTP**: Al estar el código en el HTML, el navegador no necesita hacer una petición adicional para descargar el archivo de imagen. Esto mejora el tiempo de carga, especialmente si tienes muchos iconos pequeños.

- **Accesibilidad y Semántica**: Puedes añadir elementos como `<title>` y `<desc>` dentro del SVG para mejorar la accesibilidad, proporcionando un contexto que los lectores de pantalla pueden interpretar.

7.2 Audio y Video

HTML5 introdujo los elementos `<audio>` y `<video>` para la reproducción nativa de contenido multimedia sin depender de plugins de terceros.

7.2.1 Elemento `<audio>`

Permite integrar y reproducir archivos de sonido.

HTML

```html
<audio controls preload="metadata">
  <source src="musica.mp3" type="audio/mpeg">
  <source src="musica.ogg" type="audio/ogg">
  <p>Tu navegador no soporta el elemento de audio.</p>
</audio>
```

7.2.2 Elemento `<video>`

Permite integrar, reproducir y personalizar la visualización de videos.

HTML

```html
<video width="640" height="360" controls poster="imagen-preview.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  <track src="subtitulos-es.vtt" kind="subtitles" srclang="es" label="Español">
  <p>Tu navegador no soporta el elemento de video.</p>
</video>
```

7.2.3 Atributos Comunes de Audio y Video

- `controls`: Muestra los controles de reproducción nativos del navegador (play, pausa, volumen, etc.).
- `autoplay`: Inicia la reproducción automáticamente al cargar la página (su uso debe ser cuidadoso y a menudo requiere el atributo `muted`).
- `muted`: Silencia el audio por defecto.
- `loop`: Vuelve a reproducir el medio desde el principio una vez que finaliza.
- `preload`: Sugiere al navegador cómo debe cargar el archivo (`none`, `metadata`, `auto`).
- `poster` (solo video): Muestra una imagen mientras el video se descarga o hasta que el usuario lo reproduce.

7.2.4 Pistas de Texto con `<track>`

El elemento `<track>` se usa dentro de `<video>` para añadir subtítulos, leyendas, descripciones o capítulos en formato WebVTT (.vtt).

- `kind`: Define el tipo de pista (`subtitles`, `captions`, `descriptions`, `chapters`, `metadata`).
- `srclang`: Especifica el idioma de la pista.
- `label`: Un título legible para la pista que se muestra al usuario.

7.3 Contenido Embebido con IFrames

Los iframes (Inline Frames) son elementos HTML que permiten incrustar un documento HTML completo (y por extensión, contenido externo como videos, mapas o publicidad) dentro de otro.

7.3.1 Sintaxis y Atributos Principales

HTML

```html
<iframe
  src="https://ejemplo.com/pagina-externa"
  width="800"
  height="600"
  title="Descripción accesible del contenido del iframe"
  frameborder="0"
  loading="lazy"
  allow="autoplay; encrypted-media"
  allowfullscreen
  sandbox="allow-scripts allow-same-origin">
  <p>Tu navegador no soporta iframes. Puedes ver el contenido en <a href="pagina.html">este
enlace</a>.</p>
</iframe>
```

- `src`: La URL del documento a incrustar.
- `title`: Proporciona una descripción del contenido para mejorar la accesibilidad.
- `width` / `height`: Dimensiones del frame.
- `loading="lazy"`: Difiere la carga del iframe hasta que esté cerca de ser visible.
- `allow`: Define qué características del navegador (como `autoplay` o `fullscreen`) puede usar el iframe.

7.3.2 Casos de Uso Comunes

- **Videos de plataformas**: Incrustar reproductores de YouTube o Vimeo.
- **Mapas interactivos**: Incrustar mapas de Google Maps u otros servicios.
- **Contenido de redes sociales**: Embeber publicaciones, widgets o comentarios.

7.3.3 Seguridad en IFrames

Los iframes deben usarse con precaución. La **Política del Mismo Origen (Same-Origin Policy)** restringe cómo los documentos de diferentes orígenes pueden interactuar entre sí.

7.3.3.1 El atributo `sandbox`

Es la herramienta de seguridad más importante para un iframe. Restringe las acciones que puede realizar el contenido incrustado, ayudando a prevenir ataques como el *clickjacking*.

Valores comunes:

- `allow-scripts`: Permite la ejecución de JavaScript.
- `allow-same-origin`: Permite que el contenido sea tratado como si proviniera del mismo origen que la página principal.
- `allow-forms`: Permite el envío de formularios.
- `allow-popups`: Permite la apertura de ventanas emergentes.

7.3.4 Comunicación Segura con `postMessage`

Para comunicar de forma segura una página con su iframe (y viceversa) sin violar las políticas de seguridad, se utiliza la API `postMessage`.

Página padre enviando un mensaje:

JAVASCRIPT

```javascript
const iframe = document.querySelector('iframe');
iframe.contentWindow.postMessage('Hola desde la página padre', 'https://origen-del-iframe.com')
;
```

Iframe recibiendo el mensaje:

JAVASCRIPT

```javascript
window.addEventListener('message', function(event) {
  // Siempre verificar el origen por seguridad
  if (event.origin !== 'https://origen-permitido.com') return;
  console.log('Mensaje recibido:', event.data);
});
```

7.4 Mejores Prácticas Generales

7.4.1 Rendimiento

1. **Optimización de medios**: Comprime imágenes y videos para reducir su peso sin sacrificar demasiada calidad. Usa formatos modernos como WebP y WebM.
2. **Carga diferida (Lazy Loading)**: Usa `loading="lazy"` en imágenes e iframes para que solo se carguen cuando sea necesario.
3. **Dimensiones explícitas**: Siempre define `width` y `height` en imágenes, videos e iframes para evitar que la página "salte" durante la carga.
4. **Limitar iframes**: Usa iframes solo cuando sea estrictamente necesario, ya que consumen más recursos que otros elementos.

7.4.2 Accesibilidad

1. **Texto alternativo**: El atributo `alt` en las imágenes es fundamental para los lectores de pantalla.
2. **Títulos descriptivos**: Proporciona un `title` claro y descriptivo para los iframes.
3. **Subtítulos y transcripciones**: Incluye pistas de texto en los videos para personas con discapacidad auditiva.
4. **Controles y no autoplay**: Ofrece controles visibles y evita la reproducción automática de audio y video para no desorientar al usuario.

7.4.3 Experiencia de Usuario (UX)

1. **Diseño responsivo**: Asegúrate de que todo el contenido multimedia se adapte correctamente a los diferentes tamaños de pantalla, usando el elemento `<picture>` o CSS.
2. **Indicadores de carga**: Muestra un estado de carga, especialmente para iframes o videos pesados.
3. **Contenido alternativo**: Proporciona un texto o enlace de respaldo para navegadores que no soporten un elemento específico.

7.5 Alternativas Modernas a los IFrames

Aunque los iframes son útiles, existen alternativas modernas para ciertos casos de uso que pueden ofrecer mejor rendimiento y encapsulación:

- **Web Components & Shadow DOM**: Permiten crear componentes reutilizables con estilos y scripts completamente encapsulados, evitando conflictos con el resto de la página.
- **Fetch API y Carga Dinámica**: Para cargar contenido HTML de tu propio sitio, es más eficiente usar `fetch()` para obtener el HTML e inyectarlo en un `<div>`, en lugar de usar un iframe.

7.6 Ejercicios

7.6.1 Ejercicio 1: Imagen Optimizada

Inserta una imagen en una página web.

- La ruta de la imagen es `images/paisaje.jpg`.
- El texto alternativo (`alt`) debe ser "Montañas al atardecer".
- Define un **ancho** de 600 y un **alto** de 400 píxeles para evitar saltos de contenido.
- Asegúrate de que la imagen solo se cargue cuando esté cerca de ser visible en la pantalla utilizando **carga diferida**.

7.6.2 Ejercicio 2: Imagen Responsiva con `<picture>`

Crea una estructura de imagen responsiva utilizando el elemento `<picture>`.

- Para pantallas de más de 1000px de ancho, muestra `images/desktop.jpg`.
- Para pantallas de más de 600px de ancho, muestra `images/tablet.jpg`.
- Como imagen por defecto para pantallas más pequeñas, usa `images/mobile.jpg`.
- Añade un texto alternativo apropiado para todas las versiones.

7.6.3 Ejercicio 3: Video con Subtítulos

Embebe un video en tu página utilizando la etiqueta `<video>`.

- Debe mostrar los **controles** de reproducción nativos.
- Añade una imagen de vista previa (`preview.jpg`) usando el atributo **`poster`**.
- Proporciona dos formatos de video para mayor compatibilidad: `videos/mi-video.mp4` y `videos/mi-video.webm`.
- Incluye una pista de **subtítulos** en español (`<track>`) desde el archivo `subs/es.vtt`.

7.6.4 Ejercicio 4: Iframe Seguro

Incrusta un mapa de Google Maps (o cualquier otra página externa) usando un `<iframe>`.

- La URL a incrustar es `https://www.google.com/maps/embed/...`.
- Añade un **título** descriptivo para la accesibilidad, como "Mapa interactivo de la ubicación".
- Habilita la **carga diferida**.
- Por seguridad, restringe sus permisos con el atributo **`sandbox`**, permitiendo únicamente la ejecución de scripts (`allow-scripts`).

7.7 Soluciones

7.7.1 Solución Ejercicio 1

HTML

```html
<img
  src="images/paisaje.jpg"
  alt="Montañas al atardecer"
  width="600"
  height="400"
  loading="lazy">
```

7.7.2 Solución Ejercicio 2

HTML

```html
<picture>
  <source media="(min-width: 1000px)" srcset="images/desktop.jpg">
  <source media="(min-width: 600px)" srcset="images/tablet.jpg">
  <img src="images/mobile.jpg" alt="Paisaje que se adapta al tamaño de la pantalla.">
</picture>
```

7.7.3 Solución Ejercicio 3

HTML

```html
<video width="640" height="360" controls poster="preview.jpg">
  <source src="videos/mi-video.mp4" type="video/mp4">
  <source src="videos/mi-video.webm" type="video/webm">
  <track src="subs/es.vtt" kind="subtitles" srclang="es" label="Español">
  <p>Tu navegador no soporta la reproducción de video.</p>
</video>
```

7.7.4 Solución Ejercicio 4

HTML

```html
<iframe
  src="https://www.google.com/maps/embed/..."
  width="600"
  height="450"
  title="Mapa interactivo de la ubicación"
  loading="lazy"
  sandbox="allow-scripts">
  <p>Tu navegador no soporta iframes.</p>
</iframe>
```

7.8 Referencias y Recursos

7.8.1 Documentación Oficial

- MDN Web Docs - Multimedia and embedding
- MDN Web Docs - The Inline Frame element
- HTML Living Standard - The iframe element

8 Optimización del Rendimiento

La optimización del rendimiento es un paso crucial para garantizar que un sitio web cargue rápidamente y ofrezca una buena experiencia de usuario. Muchas de estas optimizaciones se pueden implementar directamente desde el código HTML, gestionando cómo y cuándo se cargan los recursos más pesados.

8.1 Optimización de Imágenes

Las imágenes suelen ser los archivos que más impactan en el tiempo de carga de una página. Optimizarlas es fundamental para un buen rendimiento.

8.1.1 Carga Diferida (Lazy Loading)

La carga diferida indica al navegador que no cargue una imagen hasta que esté a punto de aparecer en la pantalla del usuario. Esto reduce drásticamente el tiempo de carga inicial de la página. Se implementa con el atributo `loading="lazy"`.

HTML

```html
<img src="imagen-pesada.jpg" alt="Una imagen muy grande." loading="lazy" width="800" height="600">
```

8.1.2 Formatos Modernos con el Elemento `<picture>`

Formatos de imagen como **WebP** o **AVIF** ofrecen una compresión mucho mejor que los tradicionales JPG o PNG, resultando en archivos más pequeños con una calidad similar. El elemento `<picture>` permite al navegador elegir el formato más moderno que soporte, y si no soporta ninguno, utilizará la imagen de respaldo en la etiqueta `<img>`.

HTML

```html
<picture>
  <source srcset="imagen.avif" type="image/avif">
  <source srcset="imagen.webp" type="image/webp">
  <img src="imagen.jpg" alt="Descripción de la imagen" width="1000" height="750">
</picture>
```

8.1.3 Especificar Dimensiones (`width` y `height`)

Definir siempre los atributos `width` y `height` en las etiquetas `<img>` permite que el navegador reserve el espacio exacto para la imagen antes de que se cargue. Esto evita los "saltos de contenido" (Layout Shifts) y mejora la estabilidad visual de la página.

HTML

```html
<img src="logo.png" alt="Logo de la empresa">

<img src="logo.png" alt="Logo de la empresa" width="200" height="50">
```

8.2 Optimización de Contenido Embebido

Los `<iframe>` y los vídeos son componentes muy pesados. Su carga también debe ser gestionada con cuidado.

8.2.1 Carga Diferida para `<iframe>`

Al igual que las imágenes, los `<iframe>` (usados para mapas, vídeos de YouTube, etc.) pueden cargarse de forma diferida usando `loading="lazy"`. Esto es especialmente útil para contenido que no es visible inmediatamente al cargar la página.

HTML

```html
<iframe src="https://www.google.com/maps/embed?..."
  width="600" height="450"
  loading="lazy"
  title="Mapa de ubicación">
</iframe>
```

8.2.2 Uso del Atributo `poster` en Vídeos

Para los vídeos cargados con la etiqueta `<video>`, el atributo `poster` muestra una imagen de previsualización mientras el vídeo se descarga. Esto mejora la percepción de velocidad, ya que el usuario ve una imagen relevante de inmediato en lugar de un recuadro negro.

HTML

```html
<video controls width="640" height="360" poster="preview-video.jpg">
  <source src="mi-video.mp4" type="video/mp4">
  <p>Tu navegador no soporta vídeos HTML5.</p>
</video>
```

8.3 Optimización de la Carga de Recursos (CSS y JS)

La forma en que se cargan los archivos CSS y JavaScript puede bloquear el renderizado de la página, haciéndola sentir lenta.

8.3.1 Ubicación de las Etiquetas `<link>` y `<script>`

Una de las prácticas más importantes es la correcta ubicación de las etiquetas que cargan CSS y JavaScript.

- **CSS en el `<head>`**: Las hojas de estilo (`<link rel="stylesheet">`) deben cargarse siempre dentro de la etiqueta `<head>`. Esto permite que el navegador comience a aplicar los estilos mientras carga el resto de la página, evitando un "destello de contenido sin estilo" (FOUC).
- **JavaScript al final del `<body>`**: Las etiquetas `<script>` se colocan habitualmente justo antes del cierre de la etiqueta `</body>`. Como la carga de JS bloquea el análisis del HTML, colocar los scripts al final asegura que todo el contenido visible de la página se renderice primero.

HTML

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Página Optimizada</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <h1>Contenido de la página</h1>
  <p>Este contenido se mostrará rápidamente.</p>

  <script src="app.js"></script>
</body>
</html>
```

8.3.2 Carga Asíncrona de JavaScript con `async` y `defer`

Cuando no es posible mover un script al final del `<body>`, se pueden usar los atributos `async` y `defer` para evitar el bloqueo del renderizado.

- **`defer`**: El script se descarga en segundo plano y se ejecuta solo después de que todo el HTML ha sido analizado. Es la opción recomendada.
- **`async`**: El script se descarga en segundo plano y se ejecuta en cuanto termina la descarga. Útil para scripts independientes como analíticas.

8.3.3 Precarga de Recursos Críticos con `<link rel="preload">`

Si tienes un recurso que es muy importante para el renderizado inicial de la página, puedes indicarle al navegador que lo descargue con alta prioridad usando `<link rel="preload">`.

HTML

```html
<head>
  <link rel="preload" href="important-font.woff2" as="font" type="font/woff2" crossorigin>
</head>
```

8.4 Ejercicios

8.4.1 Ejercicio 1: Optimización Completa de una Imagen

Escribe el código HTML para mostrar una imagen de producto. Debes aplicar las siguientes optimizaciones:

- Usa el elemento `<picture>` para ofrecer una versión en formato **WebP** (`imagen.webp`) y una de respaldo en formato **JPG** (`imagen.jpg`).
- La imagen debe tener un **ancho** de 800 y un **alto** de 600 píxeles.
- Implementa la **carga diferida** (`lazy loading`).
- No olvides el texto **alternativo**.

8.4.2 Ejercicio 2: Optimización de Video e Iframe

Crea el código para dos elementos embebidos:

1. Un video que muestre los **controles** de reproducción y use una imagen de previsualización (`preview.jpg`) con el atributo `poster`.
2. Un `iframe` que cargue un mapa de Google. Este `iframe` debe tener unas dimensiones de 600x450, un título descriptivo y debe usar **carga diferida**.

8.4.3 Ejercicio 3: Carga Asíncrona de JavaScript

Escribe las etiquetas `<script>` para dos archivos JavaScript diferentes que se cargarán en el `<head>` de la página:

1. El script principal de la aplicación, `app.js`, que depende del DOM y debe ejecutarse después de que se analice el HTML. Usa el atributo **`defer`**.
2. Un script de seguimiento de terceros, `analytics.js`, que es independiente y puede ejecutarse en cualquier momento. Usa el atributo **`async`**.

8.4.4 Ejercicio 4: Precarga de Recursos Críticos

Tu sitio web utiliza una fuente personalizada que es fundamental para el renderizado inicial del texto. Escribe la etiqueta `<link>` necesaria para **precargar** el archivo de fuente `fuente-principal.woff2`, que se encuentra en una carpeta `/fonts`. Asegúrate de incluir los atributos `as`, `type` y `crossorigin`.

8.5 Soluciones

8.5.1 Solución Ejercicio 1

HTML

```html
<picture>
  <source srcset="imagen.webp" type="image/webp">
  <source srcset="imagen.jpg" type="image/jpeg">
  <img
    src="imagen.jpg"
    alt="Descripción del producto"
    width="800"
    height="600"
    loading="lazy">
</picture>
```

8.5.2 Solución Ejercicio 2

HTML

```html
<video controls width="640" height="360" poster="preview.jpg">
  <source src="video.mp4" type="video/mp4">
  <p>Tu navegador no soporta vídeos HTML5.</p>
</video>

<iframe
  src="http://googleusercontent.com/maps.google.com/3?..."
  width="600"
  height="450"
  loading="lazy"
  title="Mapa de nuestra ubicación">
</iframe>
```

8.5.3 Solución Ejercicio 3

HTML

```html
<head>
  <script src="app.js" defer></script>
  <script src="analytics.js" async></script>
</head>
```

8.5.4 Solución Ejercicio 4

HTML

```html
<head>
  <link
    rel="preload"
    href="/fonts/fuente-principal.woff2"
    as="font"
    type="font/woff2"
    crossorigin>
</head>
```

8.6 Referencias y Recursos

8.6.1 Documentación Oficial

- MDN Web Docs - Learn Performance: https://developer.mozilla.org/en-US/docs/Learn/Performance
- web.dev by Google - Fast load times: https://web.dev/fast/

8.6.2 Herramientas

- Google PageSpeed Insights: https://pagespeed.web.dev/
- WebPageTest: https://www.webpagetest.org/

9 Formularios

Los formularios son el corazón de la interacción en la web. Permiten a los usuarios enviarnos datos, desde una simple consulta de búsqueda hasta un complejo registro de usuario o un pedido en una tienda online. Son, en esencia, la forma en que una página web deja de ser un documento estático y se convierte en una aplicación interactiva.

En este capítulo, exploraremos a fondo cómo construir formularios en HTML. Empezaremos por su estructura fundamental, desglosaremos la gran variedad de campos de entrada (`inputs`) que HTML5 nos ofrece y, finalmente, aprenderemos a validar los datos para garantizar que sean correctos, creando así una experiencia de usuario robusta, accesible y fluida.

9.1 Estructura de un Formulario HTML

Todo formulario comienza con el elemento `<form>`, que actúa como un contenedor inteligente para todos los controles (campos de texto, botones, etc.). Este elemento es invisible para el usuario, pero es crucial para definir cómo y a dónde se enviarán los datos recopilados.

9.1.1 Atributos del Elemento `<form>`

El comportamiento de un formulario se configura a través de sus atributos. Los más importantes son:

- `action` : Define la URL del script en el servidor (por ejemplo, un archivo PHP, Python o una ruta en un framework) que recibirá y procesará los datos del formulario. Si se omite, los datos se enviarán a la URL de la página actual.
- `method` : Especifica el método HTTP que se usará para enviar los datos. Los dos valores más comunes son:
  - **`GET`** : Anexa los datos del formulario a la URL (`/procesar?nombre=juan&edad=30`). Es útil para búsquedas o filtros, donde quieres que la URL se pueda compartir. Tiene un límite

de longitud y no debe usarse para datos sensibles, ya que son visibles en la URL y en el historial del navegador.

- `POST`: Envía los datos en el cuerpo de la petición HTTP, de forma separada a la URL. Es más seguro para enviar información sensible (como contraseñas o datos personales) y no tiene límite de tamaño. Es el método preferido para cualquier acción que modifique datos en el servidor (crear, actualizar, eliminar).

- `enctype`: Define cómo se codificarán los datos antes de enviarlos. El valor por defecto es `application/x-www-form-urlencoded`. Sin embargo, cuando el formulario incluye un `<input type="file">` para subir archivos, es **imprescindible** cambiarlo a `multipart/form-data`.
- `novalidate`: Es un atributo booleano. Si está presente, deshabilita por completo la validación de formularios nativa del navegador. Esto es útil cuando quieres tomar el control total de la validación mediante JavaScript.
- `autocomplete`: Controla si el navegador debe ofrecer autocompletar los campos del formulario. Puedes establecerlo en `on` (valor por defecto) o `off`.

HTML

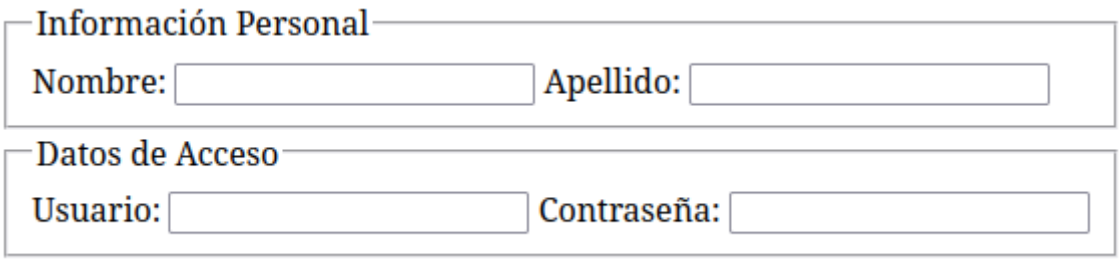
```html
<form action="/procesar-registro.php" method="post" autocomplete="off">
  </form>

<form action="/buscar" method="get">
  </form>
```

9.1.2 Agrupación de Campos con `fieldset` y `legend`

Para formularios largos o complejos, es una excelente práctica agrupar campos relacionados. Esto mejora tanto la organización visual como la accesibilidad. El elemento `<fieldset>` crea un cuadro alrededor de un grupo de controles, y `<legend>` le proporciona un título descriptivo a ese grupo.

HTML

```html
<form>
  <fieldset>
    <legend>Información Personal</legend>

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre">

    <label for="apellido">Apellido:</label>
    <input type="text" id="apellido" name="apellido">
  </fieldset>

  <fieldset>
    <legend>Datos de Acceso</legend>

    <label for="usuario">Usuario:</label>
    <input type="text" id="usuario" name="usuario">

    <label for="pass">Contraseña:</label>
    <input type="password" id="pass" name="pass">
  </fieldset>
</form>
```



9.2 Controles de Formulario: Recopilando Datos del Usuario

Los controles son las piezas interactivas que componen un formulario: campos de texto, botones, casillas, etc.

9.2.1 La Importancia de `<label>`

Las etiquetas `<label>` son fundamentales para la **accesibilidad**. Asocian un texto descriptivo con un control de formulario específico a través del atributo `for`, que debe coincidir con el `id` del control. Esto aporta dos grandes beneficios:

1. Los lectores de pantalla pueden anunciar la descripción de la etiqueta cuando el usuario se enfoca en el campo.
2. El usuario puede hacer clic en la etiqueta para activar el control, lo que aumenta el área de pulsación, algo especialmente útil en dispositivos móviles.

HTML

```html
<label for="email">Correo Electrónico:</label>
<input type="email" id="email" name="email">
```

9.2.2 Tipos de Input en HTML5

HTML5 expandió enormemente el atributo `type` del elemento `<input>`, proporcionando controles específicos para diferentes tipos de datos. Usar el tipo adecuado mejora la experiencia de usuario (mostrando teclados optimizados en móviles) y facilita la validación.

Texto Básico

- `<input type="text">`: El campo de texto genérico.
- `<input type="password">`: Similar a `text`, pero los caracteres introducidos se ocultan.
- `<textarea>`: No es un `input`, sino un elemento propio para introducir texto de múltiples líneas. Se controla con los atributos `rows` y `cols`.

HTML

```html
<label for="nombre">Nombre:</label>
<input type="text" id="nombre" name="nombre" placeholder="Escribe tu nombre">

<label for="contrasena">Contraseña:</label>
<input type="password" id="contrasena" name="contrasena" placeholder="Contraseña segura">

<label for="comentario">Comentario:</label>
<textarea id="comentario" name="comentario" rows="4" cols="50" placeholder="Escribe tu
comentario aquí..."></textarea>
```

Nombre: Escribe tu nombre Contraseña: Contraseña segura Comentario: Escribe tu comentario aquí...

Números y Rangos

- `<input type="number">`: Para valores numéricos. Permite definir `min`, `max` y `step` (incremento).
- `<input type="range">`: Representa un control deslizante (slider) para seleccionar un valor dentro de un rango.

HTML

```html
<label for="edad">Edad:</label>
<input type="number" id="edad" name="edad" min="18" max="120" step="1">

<label for="volumen">Volumen:</label>
<input type="range" id="volumen" name="volumen" min="0" max="100" value="50">
```

Edad: 20 Volumen:

Fecha y Hora

Estos tipos muestran selectores de fecha/hora nativos del navegador, muy cómodos en móviles.

- `type="date"` : Selecciona una fecha (año, mes, día).
- `type="time"` : Selecciona una hora (hora, minutos).
- `type="datetime-local"` : Combina fecha y hora, sin zona horaria.
- `type="month"` : Selecciona un mes y año.
- `type="week"` : Selecciona una semana y año.

HTML

```html
<label for="fecha-cita">Fecha de la cita:</label>
<input type="date" id="fecha-cita" name="fecha-cita">

<label for="hora-cita">Hora:</label>
<input type="time" id="hora-cita" name="hora-cita">
```

Fecha de la cita: mm / dd / yyyy Hora: 11 : 34 AM

Formatos Específicos

Estos tipos ofrecen validación de formato básica y teclados optimizados en dispositivos móviles.

- `type="email"` : Para direcciones de correo electrónico.
- `type="url"` : Para direcciones web.
- `type="tel"` : Para números de teléfono. No impone un formato, pero se puede combinar con el atributo `pattern` para ello.

HTML

```html
<label for="correo">Email:</label>
<input type="email" id="correo" name="correo" placeholder="ejemplo@dominio.com">

<label for="sitio-web">Sitio Web:</label>
<input type="url" id="sitio-web" name="sitio-web" placeholder="https://ejemplo.com">

<label for="telefono">Teléfono (9 dígitos):</label>
<input type="tel" id="telefono" name="telefono" pattern="[0-9]{9}">
```

Otros Tipos Útiles

- `type="color"`: Muestra un selector de color nativo.
- `type="file"`: Permite al usuario seleccionar uno o más archivos de su dispositivo. Usa el atributo `accept` para sugerir tipos de archivo (`.pdf` , `image/*`).
- `type="checkbox"`: Para opciones donde el usuario puede seleccionar cero, una o varias.
- `type="radio"`: Para opciones mutuamente excluyentes. Todos los radios de un mismo grupo deben compartir el mismo atributo `name`.
- `type="search"`: Semánticamente es para campos de búsqueda. Algunos navegadores le añaden una "x" para limpiar el campo.

HTML

```html
<label for="color">Elige un color:</label>
<input type="color" id="color" name="color">

<label for="documento">Adjuntar CV (.pdf):</label>
<input type="file" id="documento" name="documento" accept=".pdf">

<input type="checkbox" id="acepto" name="acepto" value="si">
<label for="acepto">Acepto los términos</label>

<p>Género:</p>
<input type="radio" id="g-masc" name="genero" value="masculino">
<label for="g-masc">Masculino</label>
<input type="radio" id="g-fem" name="genero" value="femenino">
<label for="g-fem">Femenino</label>
```

Elige un color: Adjuntar CV (.pdf): Browse... No file selected. Acepto los términos

Género:

Masculino Femenino

9.2.3 Menús Desplegables con `<select>`

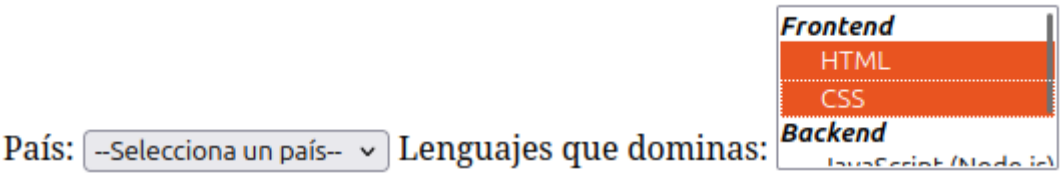
El elemento `<select>` crea una lista desplegable. Cada opción se define con un elemento `<option>`. El atributo `value` del `<option>` seleccionado es lo que se envía al servidor.

- **`multiple`**: Atributo booleano que permite seleccionar varias opciones (normalmente manteniendo pulsada la tecla `Ctrl` o `Cmd`).

- `<optgroup>`: Permite agrupar opciones relacionadas dentro del desplegable bajo una etiqueta (definida con el atributo `label`).

HTML

```html
<label for="pais">País:</label>
<select id="pais" name="pais">
  <option value="">--Selecciona un país--</option>
  <option value="es">España</option>
  <option value="mx">México</option>
</select>

<label for="lenguajes">Lenguajes que dominas:</label>
<select id="lenguajes" name="lenguajes" multiple>
  <optgroup label="Frontend">
    <option value="html">HTML</option>
    <option value="css">CSS</option>
  </optgroup>
  <optgroup label="Backend">
    <option value="js">JavaScript (Node.js)</option>
    <option value="py">Python</option>
  </optgroup>
</select>
```



9.2.4 Sugerencias con `<datalist>`

Un `<datalist>` es un híbrido entre un campo de texto y un desplegable. Proporciona una lista de sugerencias predefinidas a un `<input type="text">`, pero el usuario sigue teniendo la libertad de escribir un valor diferente. Se vincula a un `input` mediante el `id` del `datalist` y el atributo `list` del `input`.

HTML

```html
<label for="navegador">Elige tu navegador (o escribe otro):</label>
<input type="text" id="navegador" name="navegador" list="lista-navegadores">
<datalist id="lista-navegadores">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

Elige tu navegador (o escribe otro): s

Safari

9.2.5 Botones

Los botones son el disparador de las acciones del formulario. Se pueden crear con `<button>` o con `<input>`. La etiqueta `<button>` es más flexible, ya que puede contener HTML (como imágenes o texto con formato).

- `type="submit"`: El botón por defecto. Al pulsarlo, envía el formulario a la URL del `action`.
- `type="reset"`: Restablece todos los controles del formulario a sus valores iniciales.
- `type="button"`: Un botón genérico sin comportamiento predefinido. Es un lienzo en blanco para añadirle funcionalidad con JavaScript.

HTML

```html
<button type="submit">Enviar Formulario</button>
<input type="submit" value="Enviar">

<button type="reset">Limpiar Formulario</button>
<input type="reset" value="Limpiar">

<button type="button">Mostrar Ayuda</button>
<input type="button" value="Acción">
```

9.2.6 Atributos Comunes de los Controles

Existen múltiples atributos para refinar el comportamiento de los controles:

- **Básicos**:
  - `name`: Identificador clave-valor para los datos que se envían al servidor. **Esencial**.
  - `id`: Identificador único en la página, usado para asociar `label` y para seleccionar con CSS/JS.
  - `value`: El valor inicial del control. En checkboxes y radios, es el valor que se envía si está seleccionado.
  - `placeholder`: Texto de ayuda que aparece dentro del campo cuando está vacío. **No sustituye a un `<label>`**.
  - `disabled`: Deshabilita el control. No se puede interactuar con él y su valor no se envía.
  - `readonly`: El usuario no puede modificar el valor, pero sí se envía con el formulario.
- **Validación**: `required`, `minlength`, `maxlength`, `min`, `max`, `step`, `pattern`. Los veremos en detalle a continuación.
- **Autocompletado y Ayudas Móviles**: `autocomplete`, `autocapitalize`, `autocorrect`, `spellcheck`. Ayudan a mejorar la experiencia de escritura, especialmente en móviles.

9.3 Validación de Formularios

La validación es el proceso de asegurar que los datos introducidos por el usuario son correctos, completos y coherentes antes de enviarlos. Es una capa de defensa crucial para la integridad de nuestros datos.

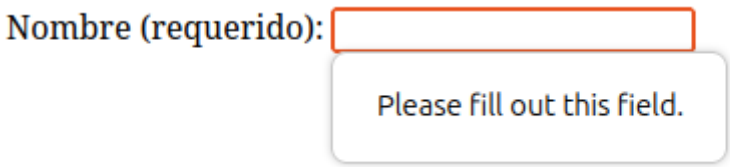
9.3.1 Validación Nativa con HTML5

HTML5 nos regala una potente validación del lado del cliente sin una sola línea de JavaScript. Simplemente añadiendo atributos a nuestros inputs, el navegador se encarga de comprobarlos antes de permitir el envío del formulario.

Campos Requeridos El atributo booleano `required` obliga al usuario a rellenar un campo. Si intenta enviar el formulario con un campo `required` vacío, el navegador mostrará un aviso.

HTML

```html
<label for="nombre-req">Nombre (requerido):</label>
<input type="text" id="nombre-req" name="nombre" required>
```



Validación por Tipo de Input Como vimos, ciertos tipos como `email` y `url` llevan incorporada una validación de formato. El navegador comprobará si el texto introducido se parece a un email o a una URL.

HTML

```html
<label for="email-req">Email (requerido):</label>
<input type="email" id="email-req" name="email" required>
```

Atributos de Restricción

- `minlength` y `maxlength`: Definen la longitud mínima y máxima para inputs de texto.
- `min` y `max`: Establecen los valores numéricos mínimo y máximo para inputs de tipo `number`, `range` y los de fecha/hora.

- `pattern`: El más potente. Especifica una **Expresión Regular** que el valor del campo debe cumplir. Es ideal para formatos personalizados (códigos postales, nombres de usuario, etc.). Usa el atributo `title` para describir el formato esperado al usuario.

HTML

```html
<label for="pass-pattern">Contraseña:</label>
<input type="password" id="pass-pattern" required
       pattern="(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"
       title="Debe contener al menos un número, una letra mayúscula y minúscula, y 8 o más
caracteres">

<label for="cp">Código Postal:</label>
<input type="text" id="cp" name="cp" required
       pattern="[0-9]{5}"
       title="El código postal debe tener 5 dígitos">
```

9.3.2 Feedback Visual con CSS

Podemos y debemos dar una respuesta visual instantánea al usuario sobre la validez de los datos. CSS nos ofrece pseudoclases que reaccionan al estado de la validación.

CSS

```css
/* Estilo base para los inputs */
input {
    border: 2px solid #ccc;
}

/* Estilo para un campo cuando su valor es válido */
input:valid {
    border-color: green;
}

/* Estilo para un campo cuando su valor es inválido y el usuario ha interactuado con él */
input:invalid:not(:placeholder-shown) {
    border-color: red;
}

/* Estilo más específico para un campo requerido que está vacío y enfocado */
input:required:invalid:focus {
    border-color: orange;
}
```

9.3.3 Validación Avanzada con JavaScript

Para un control total, mensajes de error personalizados y una experiencia más dinámica, usamos JavaScript y la **API de Validación de Restricciones (Constraint Validation API)**.

> ***Nota para principiantes:*** *Esta sección requiere conocimientos básicos de JavaScript (variables, funciones y eventos). Si aún no has aprendido JS, no te preocupes si no entiendes el código a la primera. Lo importante ahora es que sepas que* ***HTML por sí solo no basta*** *para una validación profesional completa.*

Propiedades de `validity` Cada control de formulario tiene una propiedad `validity` (`input.validity`), que es un objeto `ValidityState`. Este objeto contiene varias propiedades booleanas que describen el estado de validez del campo:

- `valueMissing`: `true` si un campo `required` está vacío.
- `typeMismatch`: `true` si el valor no coincide con el formato del `type` (ej. `email`, `url`).
- `patternMismatch`: `true` si el valor no cumple con el atributo `pattern`.
- `tooShort` / `tooLong`: `true` si el texto no cumple con `minlength` o `maxlength`.
- `rangeUnderflow` / `rangeOverflow`: `true` si un número está por debajo de `min` o por encima de `max`.
- `valid`: `true` si todas las reglas de validación se cumplen. Es la propiedad resumen.

Ejemplo de Validación en Tiempo Real

Este script escucha el evento `input` en cada campo y muestra un mensaje de error personalizado en un `<span>` adjunto, que deberías colocar en tu HTML.

HTML

```html
<label for="email-js">Email:</label>
<input type="email" id="email-js" name="email" required>
<span class="error-message" aria-live="polite"></span>
```

JAVASCRIPT

```javascript
document.addEventListener('DOMContentLoaded', () => {
    const inputs = document.querySelectorAll('input, textarea, select');

    inputs.forEach(input => {
        // Escucha cambios en tiempo real
        input.addEventListener('input', () => {
```

```
            // Encuentra el span de error más cercano
            const errorSpan = input.nextElementSibling;
            if (errorSpan && errorSpan.classList.contains('error-message')) {
                if (input.validity.valid) {
                    // Si es válido, borra el mensaje
                    errorSpan.textContent = '';
                } else {
                    // Si es inválido, muestra el error
                    mostrarError(input, errorSpan);
                }
            }
        });
    });
});

function mostrarError(input, errorSpan) {
    if (input.validity.valueMissing) {
        errorSpan.textContent = 'Este campo es requerido.';
    } else if (input.validity.typeMismatch) {
        errorSpan.textContent = `Por favor, introduce un formato de ${input.type} válido.`;
    } else if (input.validity.tooShort) {
        errorSpan.textContent = `Debe tener al menos ${input.minLength} caracteres; has
introducido ${input.value.length}.`;
    } else if (input.validity.patternMismatch) {
        // Muestra el mensaje de ayuda del atributo 'title'
        errorSpan.textContent = input.title || 'El formato introducido no es correcto.';
    } else {
        errorSpan.textContent = 'El valor introducido es inválido.';
    }
}
```

9.4 Eventos de Formulario

Podemos capturar diferentes momentos del ciclo de vida de un formulario con JavaScript para crear interacciones dinámicas.

- `submit`: Se dispara en el `<form>` cuando el usuario intenta enviarlo (con un `type="submit"` o pulsando Enter). Podemos interceptarlo para realizar una validación final con JavaScript o para enviar los datos con AJAX.
- `reset`: Se activa en el `<form>` cuando se reinicia con un `type="reset"`.
- `input`: Ocurre **inmediatamente** después de que el valor de `<input>`, `<select>` o `<textarea>` cambie. Ideal para validaciones en tiempo real.
- `change`: Similar a `input`, pero se dispara solo cuando el elemento **pierde el foco** (`blur`) después de que su valor ha cambiado.
- `focus`: Se activa cuando un elemento recibe el foco (el usuario hace clic o navega a él).
- `blur`: Se activa cuando un elemento pierde el foco.

9.5 Mejores Prácticas

1. **Accesibilidad Primero**: Usa siempre `<label>` para cada control. Agrupa campos relacionados con `<fieldset>` y `<legend>`. Asegúrate de que los mensajes de error sean accesibles, por ejemplo, usando atributos como `aria-live="polite"` para que los lectores de pantalla los anuncien.
2. **Validación en Capas**: Implementa la validación en tres niveles.
   - **HTML5 nativa**: Es tu primera línea de defensa, rápida y sencilla.
   - **JavaScript**: Mejora la experiencia de usuario con feedback en tiempo real y mensajes personalizados.
   - **Servidor**: **La más importante y obligatoria**. La validación en el cliente (`frontend`) puede ser desactivada o manipulada. La única validación segura es la que se realiza en el servidor (`backend`) antes de procesar o guardar los datos.
3. **Experiencia de Usuario (UX)**: Elige siempre el tipo de `input` más apropiado para el dato solicitado. Usa `placeholder` como una pista, no como una etiqueta. Proporciona mensajes de error claros, amables y que guíen al usuario hacia la solución.
4. **Optimización para Móviles**: Usar `type="tel"` , `type="email"` , `type="number"` , etc., mostrará el teclado más adecuado en dispositivos móviles, reduciendo la fricción. Asegúrate de que los controles y los botones sean lo suficientemente grandes para ser pulsados fácilmente con el dedo.

9.6 Ejercicios

9.6.1 Ejercicio 1: Estructura Básica y Agrupación

Crea un formulario de contacto básico.

- Debe estar agrupado dentro de un **`<fieldset>`** con una leyenda **`<legend>`** que diga "Información de Contacto".

- Incluye dos campos de texto: uno para el "Nombre" y otro para el "Email".
- Cada campo debe tener su correspondiente `<label>` correctamente asociado mediante los atributos `for` e `id`.

9.6.2 Ejercicio 2: Tipos de Input Variados

Construye un formulario de registro que pida los siguientes datos, utilizando el tipo de `input` más apropiado para cada caso:

- Un campo para la **fecha de nacimiento** (`type="date"`).
- Un campo para la **cantidad de hijos** (`type="number"`), que no permita valores negativos (`min="0"`).
- Dos botones de tipo **radio** para seleccionar el "Nivel de Estudios", con las opciones "Básico" y "Superior". Ambos deben compartir el mismo atributo `name`.
- Una casilla de verificación `checkbox` para "Aceptar términos y condiciones".

9.6.3 Ejercicio 3: Menú Desplegable y Botones

Crea un formulario para solicitar una cita.

- Incluye un menú desplegable `<select>` para que el usuario elija un servicio. Agrupa las opciones con `<optgroup>`:
  - Grupo "Consultoría": Opciones "SEO", "Marketing".
  - Grupo "Desarrollo": Opciones "Web", "Móvil".
- Añade un campo de texto largo `<textarea>` para que el usuario pueda dejar un comentario.
- Incluye un botón para **enviar** (`type="submit"`) el formulario y otro para **limpiar** (`type="reset"`) los campos.

9.6.4 Ejercicio 4: Validación Nativa con HTML5

Toma el formulario del Ejercicio 1 y añádele las siguientes reglas de **validación nativa**:

- El campo "Nombre" debe ser **requerido** (`required`) y tener una longitud mínima de 3 caracteres (`minlength="3"`).

- El campo "Email" debe ser **requerido** y, por su tipo, el navegador validará que tenga un formato de email válido.
- Añade un campo para "Nombre de usuario" que sea requerido y que solo acepte letras minúsculas y números, sin espacios, usando el atributo `pattern`.

9.7 Soluciones

9.7.1 Solución Ejercicio 1

HTML

```html
<form action="/procesar-contacto" method="post">
  <fieldset>
    <legend>Información de Contacto</legend>

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre">

    <label for="email">Email:</label>
    <input type="email" id="email" name="email">
  </fieldset>
</form>
```

9.7.2 Solución Ejercicio 2

HTML

```html
<form action="/registro" method="post">
  <label for="fecha_nacimiento">Fecha de nacimiento:</label>
  <input type="date" id="fecha_nacimiento" name="fecha_nacimiento">

  <label for="hijos">Cantidad de hijos:</label>
  <input type="number" id="hijos" name="hijos" min="0">

  <p>Nivel de Estudios:</p>
  <input type="radio" id="estudios_basico" name="nivel_estudios" value="basico">
  <label for="estudios_basico">Básico</label>

  <input type="radio" id="estudios_superior" name="nivel_estudios" value="superior">
  <label for="estudios_superior">Superior</label>

  <br><br>

  <input type="checkbox" id="terminos" name="terminos" value="aceptados">
  <label for="terminos">Aceptar términos y condiciones</label>
</form>
```

9.7.3 Solución Ejercicio 3

HTML

```html
<form action="/solicitar-cita" method="post">
  <label for="servicio">Selecciona un servicio:</label>
  <select id="servicio" name="servicio">
    <optgroup label="Consultoría">
      <option value="seo">SEO</option>
      <option value="marketing">Marketing</option>
    </optgroup>
    <optgroup label="Desarrollo">
      <option value="web">Web</option>
      <option value="movil">Móvil</option>
    </optgroup>
  </select>

  <br><br>

  <label for="comentario">Comentario:</label><br>
  <textarea id="comentario" name="comentario" rows="5" cols="40"></textarea>

  <br><br>

  <button type="submit">Enviar Cita</button>
  <button type="reset">Limpiar</button>
</form>
```

9.7.4 Solución Ejercicio 4

HTML

```html
<form action="/procesar-contacto" method="post">
  <fieldset>
    <legend>Información de Contacto</legend>

    <label for="nombre">Nombre:</label>
    <input type="text" id="nombre" name="nombre" required minlength="3">

    <label for="username">Nombre de usuario:</label>
    <input type="text" id="username" name="username" required pattern="[a-z0-9]+" title="Solo
letras minúsculas y números, sin espacios.">

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
  </fieldset>
  <button type="submit">Enviar</button>
</form>
```

9.8 Referencias y Recursos

9.8.1 Documentación Oficial

- MDN Web Docs - Guía de Formularios HTML: https://developer.mozilla.org/es/docs/Learn/Forms
- MDN Web Docs - Validación de Formularios: https://developer.mozilla.org/es/docs/Learn/Forms/Form_validation
- HTML Living Standard — Forms: https://html.spec.whatwg.org/multipage/forms.html
- W3Schools - HTML Forms: https://www.w3schools.com/html/html_forms.asp

9.8.2 Tutoriales y Guías

- Web.dev - Learn Forms: https://web.dev/learn/forms/
- CSS-Tricks - Form Validation: https://css-tricks.com/form-validation-part-1-constraint-validation-html/
- FreeCodeCamp - HTML5 Input Types: https://www.freecodecamp.org/news/html-form-input-type-and-submit-button-example/

10 HTML Semántico: Estructura con Significado

El **HTML semántico** es la práctica de utilizar etiquetas HTML que no solo definen la apariencia del contenido, sino que también describen su **significado y función** dentro de la página. En lugar de usar elementos genéricos como `<div>` y `<span>` para todo, se eligen etiquetas que comunican claramente qué tipo de contenido albergan (por ejemplo, un encabezado, un artículo, una barra de navegación).

Adoptar esta práctica es fundamental para el desarrollo web moderno, ya que mejora drásticamente la **accesibilidad** para usuarios con tecnologías de asistencia, optimiza el **posicionamiento en buscadores (SEO)** y hace que el código sea mucho más **legible y fácil de mantener** para los desarrolladores.

10.1 Elementos Semánticos Principales

Estos son los pilares sobre los que se construye una página web semántica. Definen las grandes regiones de la interfaz.

10.1.1 Estructura de Página

10.1.1.1 `<header>`

Representa un grupo de ayudas introductorias o de navegación. No se limita a ser el encabezado de toda la página; también puede ser el encabezado de una sección, un artículo o cualquier otra pieza de contenido. Generalmente contiene el título principal (`<h1>`), logotipos, y a menudo, una barra de navegación principal (`<nav>`).

HTML

```html
<header>
  <h1>El Rincón del Código</h1>
  <p>Tu blog sobre desarrollo web</p>
  <nav>
    <ul>
      <li><a href="#inicio">Inicio</a></li>
      <li><a href="#servicios">Servicios</a></li>
    </ul>
  </nav>
</header>
```

10.1.1.2 `<nav>`

Define una sección cuyo propósito es proporcionar enlaces de **navegación principal**. Es crucial para que los motores de búsqueda y los lectores de pantalla identifiquen los menús principales del sitio. No debe usarse para cada grupo de enlaces; su uso está reservado para los bloques de navegación más importantes.

HTML

```html
<nav aria-label="Navegación principal">
  <ul>
    <li><a href="/inicio">Inicio</a></li>
    <li><a href="/acerca-de">Acerca de</a></li>
    <li><a href="/contacto">Contacto</a></li>
  </ul>
</nav>
```

10.1.1.3 `<main>`

Representa el **contenido dominante y principal** del `<body>` de un documento. Es el corazón de la página, el contenido que la hace única. Una regla fundamental es que **solo debe existir un elemento `<main>` por página** y no debe ser descendiente de elementos como `<article>`, `<aside>`, `<footer>` o `<header>`.

HTML

```html
<main>
  <h1>Entendiendo el Flexbox en CSS</h1>
  <article>
    <h2>La Guía Definitiva</h2>
    <p>Flexbox es un modelo de diseño unidimensional...</p>
  </article>
</main>
```

10.1.1.4 `<article>`

Encapsula una composición **independiente y autocontenida** que, en principio, podría ser distribuida o reutilizada por sí sola. Ejemplos perfectos son una publicación de un foro, un artículo de un periódico o una entrada de un blog. Generalmente, un `<article>` tendrá su propio `<header>`.

Cuándo usar `<article>`: Pregúntate: "¿Tendría sentido este contenido si lo leyera en un lector de RSS o en otro sitio web?". Si la respuesta es sí, usa `<article>`.

HTML

```html
<article>
  <header>
    <h2>Título del Artículo</h2>
    <p>Publicado el: <time datetime="2024-02-10">10 de Febrero, 2024</time></p>
  </header>
  <p>Este es el contenido del artículo, que es autocontenido y podría existir por sí solo.</p>
</article>
```

10.1.1.5 `<section>`

Define una **agrupación temática de contenido**, típicamente con un encabezado. Representa una sección genérica de un documento que no tiene un elemento semántico más específico para representarla.

Cuándo usar `<section>`: Úsalo para agrupar partes relacionadas de una página. Por ejemplo, una página de inicio podría tener secciones para "Introducción", "Servicios" y "Testimonios". A diferencia de `<article>`, el contenido de una `<section>` está intrínsecamente ligado al documento que la contiene.

HTML

```html
<section aria-labelledby="servicios-titulo">
  <h2 id="servicios-titulo">Nuestros Servicios</h2>
  <p>Ofrecemos soluciones web a medida para tu negocio.</p>
  </section>
```

10.1.1.6 `<aside>`

Representa una porción del documento cuyo contenido está **relacionado de forma tangencial** con el contenido principal que lo rodea. A menudo se presenta como una barra lateral. Ejemplos incluyen biografías del autor, enlaces relacionados, o incluso publicidad.

HTML

```html
<main>
  <article>
    </article>
  <aside>
    <h3>Artículos Relacionados</h3>
    <ul>
      <li><a href="#">Otro artículo interesante</a></li>
      <li><a href="#">Más información sobre el tema</a></li>
    </ul>
  </aside>
</main>
```

10.1.1.7 `<footer>`

Define el **pie de página** para su sección de contenido más cercana o para todo el documento. Un `<footer>` típicamente contiene información sobre el autor, datos de copyright, enlaces a documentos relacionados o a políticas de privacidad. Puede haber varios `<footer>` en una misma página.

HTML

```html
<footer>
  <p>&copy; 2024 Mi Sitio Web. Todos los derechos reservados.</p>
  <nav>
    <a href="/privacidad">Política de Privacidad</a> |
    <a href="/terminos">Términos de Uso</a>
  </nav>
</footer>
```

10.2 Estructura Semántica Completa

Este ejemplo integra los elementos anteriores para formar el esqueleto de una página web robusta y semántica.

HTML

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi Sitio Web Semántico</title>
</head>
<body>
  <header>
    <h1>Mi Sitio Web</h1>
    <nav>
      </nav>
  </header>

  <main>
    <article>
      <header>
        <h2>Artículo Principal</h2>
        <p>Por: <address class="author">Juan Pérez</address></p>
      </header>

      <section>
        <h3>Primera Sección del Artículo</h3>
        <p>Contenido temático que pertenece a este artículo...</p>
      </section>

      <section>
        <h3>Segunda Sección del Artículo</h3>
        <p>Más contenido temático relacionado...</p>
      </section>
    </article>

    <aside>
      <h3>Contenido Relacionado</h3>
      <p>Información complementaria que no es esencial para entender el artículo.</p>
    </aside>
  </main>
```

```
  <footer>
    <p>&copy; 2024 Mi Sitio Web</p>
  </footer>
</body>
</html>
```

10.3 Elementos Semánticos Adicionales

Además de los elementos estructurales, HTML5 ofrece etiquetas para describir tipos específicos de contenido en línea.

`<figure>` 10.3.1 y `<figcaption>`

Se usan para agrupar contenido referenciado, como una imagen, un diagrama o un bloque de código, junto con su **descripción o leyenda** (`<figcaption>`). El uso de estas etiquetas asocia semánticamente la leyenda con su figura, lo cual es vital para la accesibilidad.

HTML

```
<figure>
  <img src="grafico.jpg" alt="Gráfico de barras que muestra un aumento en las ventas del 20%">
  <figcaption>Gráfico de ventas del primer trimestre de 2024.</figcaption>
</figure>
```

10.3.2 `<time>`

Permite representar fechas y horas en un formato legible para máquinas. El atributo `datetime` contiene la fecha en un formato estandarizado, mientras que el texto dentro de la etiqueta se muestra al usuario. Esto ayuda a los navegadores y motores de búsqueda a interpretar fechas para funciones como recordatorios o resultados enriquecidos.

HTML

```
<p>El evento es el <time datetime="2024-03-15T20:00:00">15 de Marzo a las 8:00 PM</time>.</p>
```

10.3.3 `<mark>`

Se utiliza para **resaltar o marcar** texto que tiene una relevancia particular en un contexto específico, como por ejemplo, resaltar los términos de búsqueda de un usuario en una página de resultados. No debe usarse simplemente para fines estilísticos.

HTML

```html
<p>Tu búsqueda de "<mark>HTML semántico</mark>" arrojó 3 resultados.</p>
```

`<details>` 10.3.4 y `<summary>`

Crean un widget de revelación de información (un "acordeón") de forma nativa. El contenido dentro de `<summary>` siempre es visible y funciona como la cabecera. Al hacer clic, el resto del contenido dentro de `<details>` se muestra u oculta.

HTML

```html
<details>
  <summary>¿Qué es el HTML Semántico?</summary>
  <p>Es la práctica de usar etiquetas HTML que describen el significado y la función del
contenido que albergan.</p>
</details>
```

10.4 Mejores Prácticas

1. **Estructura Jerárquica Clara**: Utiliza los encabezados (`<h1>` a `<h6>`) en orden lógico y sin saltos (p. ej., no pases de un `<h2>` a un `<h4>`). Un único `<h1>` por página es la norma general para el título principal.
2. **Elige el Elemento más Específico**: Antes de usar un `<div>`, pregúntate si existe un elemento semántico más apropiado (`<section>`, `<article>`, `<nav>`, etc.). Reserva los `<div>` para agrupaciones puramente estilísticas.
3. **Accesibilidad (A11y)**: El HTML semántico es la base de la accesibilidad. Asegúrate de que las imágenes tengan un atributo `alt` descriptivo y utiliza atributos ARIA solo cuando los elementos semánticos nativos no sean suficientes.

4. **SEO y Motores de Búsqueda**: Una estructura semántica clara ayuda a los *crawlers* de los motores de búsqueda a entender la jerarquía y la importancia del contenido, lo que puede impactar positivamente en tu ranking.

10.5 Beneficios del HTML Semántico

1. **Accesibilidad Mejorada**: Permite a los lectores de pantalla navegar por la página de manera eficiente, saltando directamente a la navegación, al contenido principal o al pie de página, mejorando enormemente la experiencia de usuario para personas con discapacidad visual.
2. **Mejor SEO**: Los motores de búsqueda como Google utilizan la estructura semántica para identificar las partes clave de tu contenido. Una página bien estructurada tiene más probabilidades de aparecer en resultados enriquecidos (*rich snippets*).
3. **Mantenibilidad del Código**: Un código semántico es un código auto-documentado. Es mucho más fácil para un nuevo desarrollador (¡o para tu yo del futuro!) entender la estructura de una página a simple vista, sin necesidad de depender de nombres de clases como `class="main-content"`.
4. **Compatibilidad Futura**: Las páginas semánticas son más adaptables a los cambios tecnológicos. Los navegadores, dispositivos y aplicaciones futuras podrán interpretar y presentar tu contenido de maneras que aún no imaginamos, siempre que su significado esté claramente definido.

10.6 Ejercicios

10.6.1 Ejercicio 1: Reemplazar Divs por Elementos Semánticos

Toma el siguiente código HTML, que utiliza `<div>` para la estructura, y reemplaza cada `div` con el elemento **semántico** más apropiado (`<header>` , `<nav>` , `<main>` , `<footer>`).

HTML

```html
<div id="cabecera">
  <h1>Mi Blog</h1>
</div>
```

```html
<div id="navegacion">
  <ul>
    <li><a href="#">Inicio</a></li>
    <li><a href="#">Artículos</a></li>
  </ul>
</div>
<div id="contenido-principal">
  <p>Este es el contenido principal de mi página.</p>
</div>
<div id="pie-de-pagina">
  <p>Copyright 2025</p>
</div>
```

10.6.2 Ejercicio 2: Estructurar un Artículo de Blog

Crea la estructura semántica para un artículo de blog.

- Utiliza un elemento **`<article>`** como contenedor principal.
- Dentro del artículo, añade un **`<header>`** que contenga un `<h2>` para el título y un `<p>` con el elemento **`<time>`** para la fecha de publicación (ej: `datetime="2025-07-15"`).
- El cuerpo del artículo debe estar dividido en dos **`<section>`**.
- Al lado del artículo (pero dentro del `<main>`), añade un **`<aside>`** para una lista de enlaces relacionados.

10.6.3 Ejercicio 3: Uso de `figure` y `figcaption`

Muestra una imagen con una descripción utilizando la semántica correcta.

- Usa un elemento **`<figure>`** para agrupar una imagen y su descripción.
- Dentro del `<figure>`, añade una etiqueta `<img>` (puedes usar una ruta de imagen de ejemplo como `grafico.png`).
- Incluye una descripción para la imagen usando **`<figcaption>`**. El texto debe ser "Ventas anuales de la compañía".

10.6.4 Ejercicio 4: Crear una Estructura de Página Completa

Desde cero, escribe el esqueleto de una página web completa utilizando una estructura semántica. La página debe incluir, en el orden correcto, los siguientes elementos:

- Un **`<header>`** con un `<h1>` y una sección **`<nav>`**.

- Un `<main>` que contenga un `<article>`.
- Un `<footer>` con información de copyright.

10.7 Soluciones

10.7.1 Solución Ejercicio 1

HTML

```html
<header>
  <h1>Mi Blog</h1>
</header>
<nav>
  <ul>
    <li><a href="#">Inicio</a></li>
    <li><a href="#">Artículos</a></li>
  </ul>
</nav>
<main>
  <p>Este es el contenido principal de mi página.</p>
</main>
<footer>
  <p>Copyright 2025</p>
</footer>
```

10.7.2 Solución Ejercicio 2

HTML

```html
<main>
  <article>
    <header>
      <h2>Título del Artículo del Blog</h2>
      <p>Publicado el <time datetime="2025-07-15">15 de Julio, 2025</time></p>
    </header>
    <section>
      <h3>Introducción</h3>
      <p>Contenido de la introducción...</p>
    </section>
    <section>
      <h3>Desarrollo</h3>
      <p>Contenido del desarrollo del tema...</p>
    </section>
  </article>

  <aside>
    <h3>Enlaces Relacionados</h3>
    <ul>
      <li><a href="#">Otro artículo</a></li>
      <li><a href="#">Tema relacionado</a></li>
    </ul>
```

```
  </aside>
</main>
```

10.7.3 Solución Ejercicio 3

HTML

```
<figure>
  <img src="grafico.png" alt="Gráfico de barras mostrando las ventas anuales de la compañía.">
  <figcaption>Ventas anuales de la compañía.</figcaption>
</figure>
```

10.7.4 Solución Ejercicio 4

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Página Semántica de Ejemplo</title>
</head>
<body>

  <header>
    <h1>Título Principal del Sitio</h1>
    <nav>
      <ul>
        <li><a href="/pagina1">Página 1</a></li>
        <li><a href="/pagina2">Página 2</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <h2>Contenido del Artículo Principal</h2>
      <p>Aquí iría el texto y contenido del artículo...</p>
    </article>
  </main>

  <footer>
    <p>&copy; 2025 Mi Sitio Fantástico.</p>
  </footer>

</body>
</html>
```

10.8 Referencias y Recursos

10.8.1 Documentación Oficial y Guías

- **MDN Web Docs - HTML Semántico**: La referencia principal y más completa para desarrolladores web.
- **HTML Living Standard - Sections**: La especificación oficial y técnica del W3C/WHATWG.
- **Web.dev by Google - Semantic HTML**: Guías y buenas prácticas de los desarrolladores de Google.

10.8.2 Herramientas

- **W3C Markup Validation Service**: Para verificar que tu código HTML es sintácticamente correcto.
- **WAVE Web Accessibility Tool**: Para evaluar la accesibilidad de tu página e identificar mejoras.

11 Accesibilidad en HTML

La accesibilidad web (a menudo abreviada como **a11y**) es la práctica de diseñar y desarrollar sitios y aplicaciones web para que puedan ser utilizados por todas las personas, independientemente de sus capacidades físicas, neurológicas o técnicas. No se trata solo de cumplir con un estándar, sino de garantizar una experiencia de usuario inclusiva y equitativa. Es un aspecto fundamental del desarrollo web moderno y, en muchos lugares, un requisito legal.

11.1 Principios WCAG

Las **Pautas de Accesibilidad para el Contenido Web (WCAG)** son el estándar internacional para la accesibilidad web. Se basan en cuatro principios fundamentales que forman el acrónimo **POUR**. El contenido debe ser Perceptible, Operable, Comprensible y Robusto.

11.1.1 Perceptible

La información y los componentes de la interfaz de usuario deben presentarse a los usuarios de formas que puedan percibir. Esto significa que los usuarios deben poder procesar la información, no que todos los usuarios deban percibirla de la misma manera (p. ej., visualmente).

- **Alternativas textuales para contenido no textual**: Todo contenido que no sea texto (imágenes, vídeos, iconos) debe tener una alternativa textual. El ejemplo más común es el atributo `alt` en las imágenes, que describe su contenido o función a los lectores de pantalla.
- **Alternativas para contenido multimedia**: Para el contenido basado en tiempo (audio y vídeo), se deben proporcionar alternativas como subtítulos para personas con discapacidad auditiva, o audiodescripciones para personas con discapacidad visual. Las transcripciones de texto también son increíblemente útiles.
- **Contenido adaptable y distinguible**: El contenido debe poder presentarse de diferentes maneras sin perder información o estructura. Esto incluye permitir el reajuste del tamaño del texto, asegurar un contraste de color suficiente entre el texto y el fondo, y no depender únicamente del color para transmitir información.

11.1.2 Operable

Los usuarios deben poder operar la interfaz y navegar por el contenido. La interfaz no puede requerir una interacción que un usuario no pueda realizar.

- **Accesibilidad mediante teclado**: Toda la funcionalidad interactiva (enlaces, botones, campos de formulario, etc.) debe ser operable a través de un teclado. Muchos usuarios con discapacidades motoras o visuales dependen exclusivamente del teclado para navegar.
- **Tiempo suficiente para leer y usar el contenido**: Se debe dar a los usuarios el tiempo adecuado para leer y utilizar el contenido. Se debe evitar el contenido que se mueve, parpadea o se actualiza automáticamente sin que el usuario pueda controlarlo (pausar, detener u ocultar).
- **Navegación y orientación clara**: Es fundamental proporcionar mecanismos para ayudar a los usuarios a navegar, encontrar contenido y determinar dónde se encuentran. Esto incluye encabezados descriptivos, enlaces de omisión ("skip links") para saltar bloques de contenido repetitivos y un orden de enfoque lógico.

11.1.3 Comprensible

La información y el manejo de la interfaz de usuario deben ser comprensibles. El contenido no debe exceder la capacidad de comprensión del usuario.

- **Texto legible y comprensible**: El idioma del contenido debe estar definido (`<html lang="es">`), y se deben evitar jergas o lenguaje inusualmente complejo. La estructura del texto, con párrafos, listas y encabezados, facilita la comprensión.
- **Funcionamiento predecible**: Las páginas web deben aparecer y operar de manera predecible. La navegación debe ser consistente en todo el sitio, y los componentes de la interfaz no deben cambiar de contexto de forma inesperada.
- **Ayuda en la entrada de datos**: Cuando los usuarios interactúan con formularios, se les debe ayudar a evitar y corregir errores. Las instrucciones deben ser claras, las etiquetas de los campos deben ser explícitas y los mensajes de error deben ser específicos y fáciles de entender.

11.1.4 Robusto

El contenido debe ser lo suficientemente robusto como para que pueda ser interpretado de forma fiable por una amplia variedad de agentes de usuario, incluidas las tecnologías de asistencia.

- **Compatibilidad con tecnologías actuales y futuras**: El contenido debe seguir los estándares web. El uso de HTML y CSS válidos y bien formados maximiza la compatibilidad con los navegadores actuales y futuros, así como con los lectores de pantalla y otras tecnologías de asistencia.
- **Código válido y bien formado**: Un código limpio, sin errores de sintaxis y con elementos correctamente anidados, asegura que las tecnologías de asistencia puedan analizar la página y presentar una representación precisa al usuario.

11.2 ARIA Roles y Atributos

ARIA (Accessible Rich Internet Applications) es un conjunto de atributos que se pueden añadir a los elementos HTML para mejorar su accesibilidad. ARIA no añade funcionalidad, solo proporciona información semántica adicional para las tecnologías de asistencia cuando el HTML nativo no es suficiente, especialmente en componentes dinámicos y complejos como pestañas, menús desplegables o sliders.

> ***La primera regla de ARIA**: Si puedes usar un elemento HTML nativo (`<button>`, `<nav>`, `<input type="checkbox">`) que ya tiene la semántica y el comportamiento que necesitas, úsalo en lugar de un `<div>` con atributos ARIA.*

11.2.1 Roles

Los roles ARIA definen qué es un elemento o qué hace. Se dividen en varias categorías.

HTML

```html
<header role="banner">...</header>
<nav role="navigation">...</nav>
<main role="main">...</main>
<aside role="complementary">...</aside>
<footer role="contentinfo">...</footer>

<article role="article">...</article>
<section role="region">...</section>
<div role="tabpanel">...</div>

<span role="button" tabindex="0">Acción</span>
<div role="alert">Mensaje importante</div>
<div role="dialog">Ventana de diálogo</div>
```

11.2.2 Estados y Propiedades

Describen las características y el estado actual de un elemento.

- **Propiedades**: Definen características que no suelen cambiar una vez establecidas (p. ej., `aria-label`, `aria-required`).
- **Estados**: Definen condiciones que pueden cambiar dinámicamente según la interacción del usuario (p. ej., `aria-pressed`, `aria-hidden`, `aria-invalid`).

HTML

```html
<button aria-pressed="true">Notificaciones activadas</button>
<div aria-hidden="true">Contenido oculto para lectores de pantalla</div>
<input aria-invalid="true"> <button aria-label="Cerrar ventana">X</button> <input id="pwd" aria
-describedby="pwd-hint"> <div id="pwd-hint">La contraseña debe tener más de 8 caracteres.</div>
<input aria-required="true">
```

11.3 Navegación por Teclado

Una navegación por teclado fluida y lógica es crucial para la accesibilidad. El orden en que los elementos interactivos reciben el foco al presionar la tecla `Tab` debe seguir un flujo visual y lógico.

11.3.1 Orden de Tabulación

El orden de tabulación predeterminado sigue el orden de los elementos en el código fuente HTML. El atributo `tabindex` puede modificar este comportamiento.

- `tabindex="0"`: Inserta un elemento en el orden de tabulación natural. Es útil para hacer "focusable" elementos que no lo son por defecto, como un `<div>` que actúa como un botón.
- `tabindex="-1"`: Elimina un elemento del orden de tabulación, pero permite que reciba el foco mediante programación (usando JavaScript). Es útil para gestionar el foco en ventanas modales o componentes complejos.
- `tabindex="1"` (o cualquier valor mayor que cero): 🚨 **Práctica a evitar.** Define un orden de tabulación explícito. Esto es muy problemático porque rompe el flujo natural del documento y es muy frágil de mantener.

HTML

```html
<div tabindex="2">Segundo</div>
<div tabindex="1">Primero</div>
<div tabindex="3">Tercero</div>

<a href="#" tabindex="0">Enlace focusable por defecto</a>
<div role="button" tabindex="0">Div focusable en el orden natural</div>
<div id="modal" tabindex="-1">No focusable con Tab, pero sí con JS (ej. modal.focus())</div>
```

Importante: Siempre debe haber una indicación visual clara de qué elemento tiene el foco (normalmente un contorno o `outline`).

11.3.2 Atajos de Teclado

El atributo `accesskey` permite asignar un atajo de teclado a un elemento.

> ***Precaución**: Usa `accesskey` con mucho cuidado. Las combinaciones de teclas pueden entrar en conflicto con los atajos del sistema operativo o de las tecnologías de asistencia. Si lo usas, asegúrate de informar al usuario sobre los atajos disponibles.*

HTML

```html
<button accesskey="s">Guardar</button>
<a href="#main" accesskey="m">Ir al contenido principal</a>
```

11.4 Mejores Prácticas

11.4.1 Estructura Semántica

Usar los elementos HTML correctos para su propósito es la base de la accesibilidad. Un HTML semántico proporciona una estructura clara y significativa que las tecnologías de asistencia pueden interpretar.

HTML

```html
<!DOCTYPE html>
<html lang="es"> <head>
    <title>Título descriptivo y único de la página</title>
</head>
<body>
    <header>
        <h1>Encabezado Principal (solo uno por página)</h1>
        <nav>
            </nav>
    </header>

    <main> <article>
            <h2>Título del Artículo</h2>
            <p>Contenido...</p>
        </article>
    </main>

    <footer>
        <p>Información de pie de página: copyright, enlaces secundarios, etc.</p>
    </footer>
</body>
</html>
```

11.4.2 Imágenes Accesibles

Las imágenes deben tratarse de manera diferente según su propósito.

- **Informativas**: Si una imagen transmite información relevante, su `alt` debe describir concisamente esa información.
- **Decorativas**: Si una imagen es puramente ornamental y no añade información, su `alt` debe estar vacío (`alt=""`). Esto indica a los lectores de pantalla que la ignoren.
- **Complejas (gráficos, diagramas)**: Requieren un `alt` con un resumen corto y una descripción más larga cerca de la imagen, a menudo usando `<figure>` y `<figcaption>`.

HTML

```html
<a href="/"><img src="logo.png" alt="Logo de la empresa, ir a la página de inicio"></a>

<img src="borde-bonito.png" alt="">

<figure>
    <img src="grafico-ventas.png"
alt="Gráfico de barras mostrando un aumento en las ventas del último trimestre.">
    <figcaption>
        Gráfico detallado de las ventas del año 2024. El primer trimestre cerró con 1M€,
        el segundo con 1.5M€ y el tercero alcanzó los 2M€.
    </figcaption>
</figure>
```

11.4.3 Formularios Accesibles

Los formularios son un punto de interacción clave. Para que sean accesibles:

- Usa `<label for="id_del_input">` para asociar explícitamente una etiqueta a cada control de formulario.
- Agrupa controles relacionados (como un conjunto de `radio button` o `checkbox`) con `<fieldset>` y describe el grupo con `<legend>`.
- Usa atributos ARIA como `aria-required="true"` para indicar campos obligatorios y `aria-invalid="true"` para señalar errores.

HTML

```html
<form>
    <div>
        <label for="nombre">Nombre:</label>
        <input type="text" id="nombre" name="nombre" aria-required="true">
    </div>

    <div>
        <fieldset>
            <legend>¿Cómo prefieres que te contactemos?</legend>

            <input type="checkbox" id="email" name="contacto_email">
            <label for="email">Por email</label>

            <input type="checkbox" id="telefono" name="contacto_telefono">
            <label for="telefono">Por teléfono</label>
        </fieldset>
    </div>

    <button type="submit">Enviar</button>
</form>
```

11.4.4 Enlaces Descriptivos

El texto de un enlace debe describir claramente a dónde lleva. Esto es vital para usuarios de lectores de pantalla que a menudo navegan listando los enlaces de una página.

- **Evita textos genéricos**: "Haz clic aquí", "Leer más" o "Ver" no tienen sentido fuera de su contexto visual.
- **Sé específico**: El texto del enlace debe ser suficiente para entender el destino.

- **Informa sobre el tipo de archivo**: Si el enlace apunta a un archivo (PDF, DOCX), es una buena práctica indicarlo para que el usuario sepa qué esperar.

HTML

```html
<a href="catalogo-verano-2024.html">Ver catálogo de productos de verano 2024</a>

<a href="catalogo-verano-2024.html">Click aquí</a>

<a href="informe-anual.pdf" aria-label="Descargar informe anual 2024 (PDF, 2.5MB)">
    Informe anual 2024
</a>
```

11.5 Lista de Verificación de Accesibilidad

1. **Estructura HTML**
   - ¿El código usa HTML semántico (`<header>` , `<nav>` , `<main>` , `<footer>` , etc.)?
   - ¿La jerarquía de encabezados (`<h1>` a `<h6>`) es lógica y no salta niveles?
   - ¿Se ha definido el idioma de la página con `lang` ?
   - ¿El título (`<title>`) de la página es único y descriptivo?
2. **Contenido**
   - ¿Todas las imágenes informativas tienen un texto `alt` descriptivo?
   - ¿Las imágenes decorativas tienen un `alt` vacío (`alt=""`)?
   - ¿El contraste de color entre el texto y el fondo cumple al menos el ratio 4.5:1 (Nivel AA)?
   - ¿La información no se transmite únicamente a través del color?
3. **Interacción**
   - ¿Se puede acceder y operar toda la funcionalidad (enlaces, botones, formularios) usando solo el teclado?
   - ¿El elemento que tiene el foco del teclado es visualmente evidente?

- ¿El orden de tabulación es lógico y sigue el flujo visual de la página?
- ¿El contenido que se mueve o parpadea puede ser detenido por el usuario?

4. **Multimedia**

- ¿Los vídeos tienen subtítulos sincronizados y precisos?
- ¿Se proporcionan transcripciones textuales para el contenido de audio y vídeo?
- ¿Los reproductores de multimedia son accesibles mediante teclado?

11.6 Ejercicios

11.6.1 Ejercicio 1: Alternativas para Imágenes

Escribe el código `<img>` para los siguientes dos casos, aplicando la práctica correcta para el atributo `alt`:

1. Una imagen que es el logo de una empresa y enlaza a la página de inicio. El logo dice "Empresa Global".
2. Una imagen que es puramente decorativa, como una línea divisoria con un diseño floral.

11.6.2 Ejercicio 2: Formularios Accesibles

Crea un campo de formulario para una dirección de correo electrónico.

- Debe tener una `<label>` correctamente asociada.
- El campo de texto (`<input>`) debe indicar a las tecnologías de asistencia que es **requerido** usando un atributo ARIA.
- Añade un botón de envío que tenga un `aria-label` para describir su acción de forma más explícita, como "Enviar formulario de contacto".

11.6.3 Ejercicio 3: Enlaces Descriptivos

Corrige el siguiente fragmento de código para que los enlaces sean descriptivos y accesibles.

1. Reemplaza el texto "Leer más" por uno más descriptivo.
2. Añade un `aria-label` al segundo enlace para informar al usuario sobre el tipo y tamaño del archivo.

HTML

```html
<p>Nuestra empresa acaba de lanzar una nueva iniciativa verde. <a href="/noticia.html">Leer más</a></p>

<p>Puedes consultar los resultados del último trimestre. <a href="/reporte.pdf">Descargar reporte</a></p>
```

11.6.4 Ejercicio 4: Roles de Landmark ARIA

Estructura el esqueleto de una página web usando elementos semánticos (`<header>`, `<main>`, `<footer>`). Añade los **roles de landmark ARIA** correspondientes a cada una de estas secciones para asegurar la máxima compatibilidad con tecnologías de asistencia más antiguas.

11.7 Soluciones

11.7.1 Solución Ejercicio 1

HTML

```html
<a href="/index.html">
  <img src="logo.png" alt="Logo de Empresa Global - Ir a la página de inicio">
</a>

<img src="linea-floral.png" alt="">
```

11.7.2 Solución Ejercicio 2

HTML

```html
<form action="/suscripcion" method="post">
  <label for="email">Correo Electrónico:</label>
  <input type="email" id="email" name="email" aria-required="true">
```

```
  <button type="submit" aria-label="Enviar formulario de suscripción">Suscribirse</button>
</form>
```

11.7.3 Solución Ejercicio 3

HTML

```
<p>Nuestra empresa acaba de lanzar una nueva iniciativa verde. <a href="/noticia.html">Leer la
noticia completa sobre la iniciativa verde</a></p>

<p>Puedes consultar los resultados del último trimestre. <a href="/reporte.pdf" aria-label="De
scargar reporte trimestral en formato PDF (5MB)">Descargar reporte</a></p>
```

11.7.4 Solución Ejercicio 4

HTML

```
<body>
  <header role="banner">
    <h1>Mi Sitio Web Accesible</h1>
  </header>

  <main role="main">
    <p>Contenido principal de la página...</p>
  </main>

  <footer role="contentinfo">
    <p>&copy; 2025 Mi Sitio Web.</p>
  </footer>
</body>
```

11.8 Referencias y Recursos

11.8.1 Documentación Oficial

- WCAG 2.1
- WAI-ARIA

- MDN Accessibility

11.8.2 Herramientas

- WAVE Web Accessibility Tool
- aXe DevTools
- Lighthouse

11.8.3 Guías y Tutoriales

- WebAIM
- A11Y Project
- Inclusive Components

12 Elementos Interactivos en HTML

Los elementos interactivos en HTML son componentes que permiten a los usuarios interactuar con el contenido de una página web, creando interfaces más dinámicas y ricas sin depender exclusivamente de JavaScript. HTML5 introdujo y estandarizó varios elementos que facilitan la creación de estas interacciones de forma nativa, mejorando la accesibilidad y el rendimiento.

12.1 details y summary

El elemento `<details>` es un widget de revelación que permite mostrar u ocultar un bloque de información. Funciona como un acordeón nativo. Por defecto, la información está oculta. El usuario puede hacer clic en el encabezado para mostrarla.

El elemento `<summary>`, que debe ser el primer hijo de `<details>`, proporciona el **encabezado o etiqueta visible** para el widget. Al hacer clic en el `<summary>`, el estado del `<details>` padre se alterna entre abierto y cerrado.

Este par de etiquetas es ideal para secciones de Preguntas Frecuentes (FAQ), menús desplegables o cualquier contenido que deba estar disponible sin ocupar espacio visual inicialmente.

12.1.1 Uso Básico

Para que funcione, solo necesitas anidar un `<summary>` y el contenido que desees dentro de una etiqueta `<details>`. El navegador se encarga del resto.

HTML

```html
<details>
    <summary>Haz clic para ver más información</summary>
    <p>Este es el contenido que se mostrará al hacer clic.</p>
</details>
```

12.1.2 Atributo open

Si deseas que el contenido sea visible por defecto al cargar la página, puedes añadir el atributo booleano `open`.

HTML

```html
<details open>
    <summary>Esta sección está abierta por defecto</summary>
    <p>Gracias al atributo 'open', este texto es visible desde el principio.</p>
</details>
```

12.2 dialog

El elemento `<dialog>` representa una **ventana de diálogo** o componente modal que se superpone al contenido principal de la página. Es extremadamente útil para notificaciones, formularios, o solicitar una acción específica del usuario sin que abandone el contexto de la página actual.

Los diálogos pueden ser de dos tipos:

- **Modal (`showModal()`)**: Se abre sobre todo el contenido y bloquea la interacción con el resto de la página. Generalmente se oscurece el fondo (`::backdrop`). El usuario debe interactuar con el diálogo (por ejemplo, cerrándolo) para volver al contenido principal.
- **No modal (`show()`)**: Se abre en la página pero no impide que el usuario interactúe con otros elementos.

12.2.1 Diálogo Básico

La estructura HTML es simple. Para controlarlo (abrirlo y cerrarlo), se necesita una pequeña ayuda de JavaScript.

HTML

```html
<dialog id="miDialog">
    <h2>Título del Diálogo</h2>
    <p>Contenido del diálogo...</p>
    <button onclick="this.parentElement.close()">Cerrar</button>
</dialog>

<button onclick="document.getElementById('miDialog').showModal()">
    Abrir Diálogo
</button>
```

12.2.2 Métodos y Eventos

La interacción con el diálogo se gestiona a través de métodos y eventos de su interfaz DOM.

JAVASCRIPT

```javascript
const dialog = document.querySelector('dialog');

// Abrir como modal: bloquea el resto de la interfaz.
dialog.showModal();

// Abrir como no modal: permite interactuar con el resto de la página.
dialog.show();

// Cerrar el diálogo.
dialog.close();

// El evento 'close' se dispara cuando el diálogo se cierra.
dialog.addEventListener('close', () => {
    console.log('Diálogo cerrado. Se puede ejecutar código aquí.');
});
```

12.3 canvas

El elemento `<canvas>` actúa como un **lienzo en blanco** en la página web, sobre el cual puedes dibujar gráficos, renderizar imágenes, crear animaciones o incluso procesar vídeos en tiempo real. A

diferencia de los elementos HTML estándar, `<canvas>` es un contenedor de gráficos rasterizados (mapa de bits), lo que significa que una vez que algo se dibuja, el navegador no tiene conocimiento de qué objeto es (por ejemplo, no sabe si es un círculo o un cuadrado, solo ve píxeles).

Toda la lógica de dibujo se realiza a través de **JavaScript**, utilizando la API de Canvas.

12.3.1 Canvas Básico

Primero, defines el lienzo en HTML con un `id` y dimensiones. Luego, usas JavaScript para obtener su **contexto de renderizado** (el más común es `'2d'`), que es el objeto que contiene todos los métodos y propiedades para dibujar.

HTML

```html
<canvas id="miCanvas" width="300" height="200"></canvas>

<script>
// 1. Obtener el elemento canvas
const canvas = document.getElementById('miCanvas');
// 2. Obtener el contexto de dibujo 2D
const ctx = canvas.getContext('2d');

// 3. Dibujar en el contexto
// Dibujar un rectángulo azul
ctx.fillStyle = 'blue';
ctx.fillRect(10, 10, 100, 50); // (x, y, ancho, alto)

// Dibujar una línea
ctx.beginPath(); // Iniciar una nueva ruta
ctx.moveTo(0, 0); // Mover el "lápiz" al punto de inicio (x, y)
ctx.lineTo(300, 200); // Dibujar una línea hasta el punto final (x, y)
ctx.stroke(); // Renderizar la línea
</script>
```

12.3.2 Ejemplos de Canvas

El contexto `2d` ofrece una gran variedad de funciones para dibujar formas, texto, imágenes y aplicar estilos complejos.

12.3.2.1 Dibujar Formas y Texto

JAVASCRIPT

```javascript
// Círculo relleno
ctx.beginPath();
ctx.arc(100, 100, 50, 0, 2 * Math.PI); // (x, y, radio, anguloInicio, anguloFin)
ctx.fillStyle = 'green';
ctx.fill();

// Texto
ctx.font = '20px Arial';
ctx.fillStyle = 'black';
ctx.fillText('Hola Mundo', 10, 50); // (texto, x, y)

// Gradiente lineal
const gradient = ctx.createLinearGradient(0, 0, 200, 0); // (x1, y1, x2, y2)
gradient.addColorStop(0, 'red'); // Color al inicio
gradient.addColorStop(1, 'blue'); // Color al final
ctx.fillStyle = gradient;
ctx.fillRect(10, 120, 180, 50);
```

12.4 Drag and Drop (Arrastrar y Soltar)

La API de Drag and Drop (D&D) de HTML5 permite que los usuarios seleccionen un elemento con el ratón, lo arrastren a una nueva ubicación y lo suelten, desencadenando una acción. Esto es nativo del navegador y no requiere librerías externas.

El proceso implica dos tipos de elementos:

- **Elemento arrastrable**: El objeto que el usuario puede mover. Se marca con el atributo `draggable="true"`.
- **Zona de destino**: El área donde se puede soltar el elemento arrastrable.

12.4.1 Ciclo de Vida y Eventos Clave

1. `dragstart`: Se dispara en el elemento arrastrable cuando el usuario comienza a arrastrarlo. Aquí se especifica qué datos se están transfiriendo.
2. `dragover`: Se dispara repetidamente sobre un elemento de destino mientras se arrastra un elemento sobre él. Es **crucial** llamar a `event.preventDefault()` en este evento para permitir que se produzca el `drop`.
3. `drop`: Se dispara en el elemento de destino cuando el usuario suelta el elemento arrastrable sobre él. Aquí es donde se procesan los datos transferidos.

12.4.2 Implementación con JavaScript

HTML

```html
<div id="item" draggable="true" ondragstart="drag(event)">
    Arrástrame
</div>

<div id="destino" ondrop="drop(event)" ondragover="allowDrop(event)">
    Zona de destino
</div>
```

JAVASCRIPT

```javascript
// Permite que un elemento sea soltado aquí.
// Sin esto, el evento 'drop' nunca se dispararía.
function allowDrop(ev) {
    ev.preventDefault();
}

// Se activa cuando el usuario empieza a arrastrar el elemento.
// Guarda el ID del elemento que se está arrastrando.
function drag(ev) {
    ev.dataTransfer.setData("text/plain", ev.target.id);
}

// Se activa cuando el elemento se suelta en la zona de destino.
function drop(ev) {
    ev.preventDefault(); // Evita el comportamiento por defecto del navegador.
    const data = ev.dataTransfer.getData("text/plain"); // Obtiene el ID guardado.
    const elementoArrastrado = document.getElementById(data);
    ev.target.appendChild(elementoArrastrado); // Añade el elemento arrastrado a la zona de
destino.
}
```

12.5 Mejores Prácticas

1. **Accesibilidad**:
   - Asegúrate de que los elementos interactivos sean **manejables por teclado**. Por ejemplo, un `<details>` se puede abrir y cerrar con la barra espaciadora. Un `<dialog>` atrapa el foco del teclado.
   - Usa **roles ARIA** (`aria-*`) cuando crees componentes personalizados que imitan el comportamiento de elementos nativos para que los lectores de pantalla los entiendan.
   - Proporciona **feedback claro**, tanto visual (cambios de estilo) como auditivo (usando ARIA live regions si es necesario).
2. **Rendimiento**:
   - Optimiza las operaciones en `<canvas>`, ya que los redibujados constantes pueden ser costosos. Dibuja solo lo que ha cambiado.
   - Usa **`requestAnimationFrame`** para animaciones en lugar de `setInterval` o `setTimeout`. Sincroniza el dibujo con el ciclo de refresco del navegador, lo que resulta en animaciones más fluidas y eficientes.
   - Limita el número de elementos con `draggable="true"` en páginas complejas para evitar sobrecargar al navegador.
3. **Compatibilidad**:
   - Aunque la mayoría de los navegadores modernos soportan estas etiquetas, siempre es bueno verificar la compatibilidad en sitios como Can I use...
   - Implementa **fallbacks** o polyfills si necesitas dar soporte a navegadores muy antiguos donde estas características no existen.
4. **UX (Experiencia de Usuario)**:
   - Proporciona **indicadores visuales claros** de que un elemento es interactivo (por ejemplo, el cursor cambia a `pointer` sobre un `<summary>`).
   - Mantén la **consistencia** en el diseño y comportamiento de las interacciones a lo largo de tu sitio web.

- Ten en cuenta los **dispositivos táctiles**. Las interacciones como `drag and drop` pueden ser más complicadas en pantallas táctiles que con un ratón.

12.6 Ejercicios

12.6.1 Ejercicio 1: Acordeón de Preguntas Frecuentes

Crea una sección de preguntas frecuentes (FAQ) utilizando los elementos **`<details>`** y **`<summary>`**. Debes incluir al menos dos preguntas. El texto visible de cada pregunta estará en la etiqueta `<summary>` y la respuesta en un párrafo dentro de `<details>`.

12.6.2 Ejercicio 2: Acordeón Abierto por Defecto

Crea otro elemento **`<details>`** que, a diferencia de los anteriores, aparezca **abierto** por defecto cuando se carga la página.

12.6.3 Ejercicio 3: Estructura de un Diálogo

Aunque para abrir y cerrar un `<dialog>` de forma dinámica se necesita JavaScript, puedes escribir su estructura básica en HTML. Crea un elemento **`<dialog>`** que contenga un título `<h2>` y un párrafo `<p>`.

12.6.4 Ejercicio 4: Lienzo para Dibujo

Prepara el terreno para un futuro dibujo. Crea un elemento **`<canvas>`** con un `id` de "miLienzo", un **ancho** de 300 y un **alto** de 150. Añade un borde simple usando el atributo `style` para que sea visible en la página.

12.7 Soluciones

12.7.1 Solución Ejercicio 1

HTML

```html
<h3>Preguntas Frecuentes</h3>

<details>
  <summary>¿Cuál es el horario de atención?</summary>
  <p>Nuestro horario de atención es de lunes a viernes, de 9:00 a 18:00.</p>
</details>

<details>
  <summary>¿Aceptan devoluciones?</summary>
  <p>Sí, aceptamos devoluciones dentro de los 30 días posteriores a la compra.</p>
</details>
```

12.7.2 Solución Ejercicio 2

HTML

```html
<details open>
  <summary>Información Importante (Visible por defecto)</summary>
  <p>Este contenido es visible desde el principio porque el elemento 'details' tiene el
atributo 'open'.</p>
</details>
```

12.7.3 Solución Ejercicio 3

HTML

```html
<dialog>
  <h2>Título del Diálogo</h2>
  <p>Este es el contenido que aparecería en la ventana modal.</p>
</dialog>
```

12.7.4 Solución Ejercicio 4

HTML

```html
<canvas id="miLienzo" width="300" height="150" style="border: 1px solid black;"></canvas>
```

12.8 Referencias y Recursos

12.8.1 Documentación Oficial

- MDN Web Docs - <details>
- MDN Web Docs - <dialog>
- MDN Web Docs - Canvas API

12.8.2 Tutoriales y Guías

- HTML5 Canvas Tutorial (W3Schools)
- HTML Drag and Drop API (MDN)
- A Complete Guide to the Dialog Element (CSS-Tricks)

13 Atributos data-* en HTML

Los **atributos de datos** (comúnmente conocidos como *data attributes*) son una característica estándar de HTML5 diseñada para permitirnos **almacenar información adicional y personalizada** directamente en los elementos HTML. Su propósito principal es guardar datos privados o específicos de una página o aplicación, a los cuales podemos acceder y manipular fácilmente con JavaScript y CSS.

Estos atributos no afectan al SEO ni a la semántica del documento, ya que son ignorados por los navegadores (en términos de renderizado) y los lectores de pantalla. Son, en esencia, un puente entre tu HTML y tus scripts o estilos, permitiendo un código más limpio y organizado.

13.1 Sintaxis Básica

La sintaxis es simple y estricta para garantizar que no haya conflictos con futuros atributos estándar de HTML.

13.1.1 Definición en HTML

Todos los atributos de datos deben comenzar con el prefijo `data-`, seguido de un nombre que tú elijas.

HTML

```html
<div id="usuario-1"
     data-usuario="juan"
     data-id="123"
     data-rol="admin">
  Contenido del perfil de Juan.
</div>
```

En este ejemplo, hemos asociado un nombre de usuario, un ID y un rol directamente al `<div>`.

13.1.2 Nombres con Múltiples Palabras

Para nombres de atributos que constan de varias palabras, el estándar recomienda usar el formato **kebab-case** (palabras separadas por guiones).

HTML

```html
<article data-categoria-principal="tecnologia"
         data-fecha-publicacion="2024-02-10">
  Contenido del artículo sobre tecnología.
</article>
```

Como veremos más adelante, esta convención se traduce de manera automática y predecible a JavaScript.

13.2 La API dataset

Para interactuar con los atributos `data-*` de una manera cómoda y eficiente desde JavaScript, los elementos HTML disponen de una propiedad especial llamada `dataset`. Esta propiedad es un objeto (`DOMStringMap`) que contiene todos los atributos `data-*` de un elemento.

13.2.1 Acceso desde JavaScript

El objeto `dataset` permite leer, modificar, añadir y eliminar atributos de datos de forma muy intuitiva.

JAVASCRIPT

```javascript
// 1. Obtener el elemento del DOM
const elemento = document.querySelector('#miElemento');

// 2. Leer atributos de datos
// La propiedad en camelCase corresponde al nombre del atributo en kebab-case
const usuario = elemento.dataset.usuario; // -> "juan"
const fechaPublicacion = elemento.dataset.fechaPublicacion; // -> "2024-02-10"

// 3. Modificar o añadir nuevos atributos de datos
// Si el atributo no existe, se crea. Si ya existe, se actualiza.
elemento.dataset.estado = 'activo'; // Añade/actualiza data-estado="activo"
elemento.dataset.ultimaModificacion = '2025-07-16'; // Añade/actualiza data-ultima-
modificacion="..."

// 4. Eliminar atributos de datos
// Se usa el operador `delete` como en cualquier objeto de JavaScript.
delete elemento.dataset.temporal; // Elimina el atributo data-temporal del elemento
```

```
// 5. Verificar la existencia de un atributo
if ('id' in elemento.dataset) {
    console.log('El atributo data-id existe y su valor es:', elemento.dataset.id);
}
```

13.2.2 Conversión de Nombres: Kebab-case a CamelCase

Existe una regla de conversión automática entre el nombre del atributo en HTML y la clave del objeto en JavaScript:

- **En HTML**: Se usa `kebab-case` (`data-nombre-atributo`).
- **En JavaScript**: Se convierte a `camelCase` (`dataset.nombreAtributo`).

El navegador se encarga de esta transformación por nosotros.

HTML

```
<div id="usuario"
     data-nombre-completo="Juan Pérez"
     data-fecha-nacimiento="1990-01-01">
</div>

<script>
const usuario = document.getElementById('usuario');

// 'data-nombre-completo' se convierte en 'nombreCompleto'
console.log(usuario.dataset.nombreCompleto); // "Juan Pérez"

// 'data-fecha-nacimiento' se convierte en 'fechaNacimiento'
console.log(usuario.dataset.fechaNacimiento); // "1990-01-01"
</script>
```

13.3 Uso con CSS

Los atributos `data-*` son extremadamente útiles para aplicar estilos condicionales sin necesidad de añadir o quitar clases con JavaScript. Esto se logra mediante **selectores de atributos**.

13.3.1 Selectores de Atributos

Puedes seleccionar elementos basándote en la existencia, el valor exacto o un valor parcial de un `data attribute`.

CSS

```
/* Selecciona elementos que tengan el atributo data-tipo="importante" */
[data-tipo="importante"] {
    background-color: yellow;
    border-left: 5px solid gold;
}

/* Selecciona elementos cuyo data-categoria contenga la subcadena "tech" */
[data-categoria*="tech"] {
    color: blue;
}

/* Selecciona elementos cuyo data-rol comience con "admin" */
[data-rol^="admin"] {
    font-weight: bold;
}

/* Selecciona elementos cuyo data-fichero termine con ".pdf" */
[data-fichero$=".pdf"]::after {
    content: ' (PDF)';
}

/* Selecciona elementos con un atributo data-visible (sin importar su valor) */
[data-visible] {
    display: block;
}
```

13.3.2 Mostrar Contenido con `attr()`

Una técnica muy poderosa es usar la función `attr()` de CSS para mostrar el valor de un `data attribute` directamente en la página, generalmente con los pseudoelementos `::before` o `::after`.

HTML

```
<span data-tooltip="Este es un campo obligatorio">⚠️</span>
```

CSS

```
span[data-tooltip] {
    position: relative;
    cursor: help;
}

/* Crear un "tooltip" al pasar el ratón */
span[data-tooltip]:hover::after {
    content: attr(data-tooltip); /* Aquí se extrae el valor */
    position: absolute;
    bottom: 100%;
    left: 50%;
    transform: translateX(-50%);
    padding: 5px;
    background-color: #333;
```

```
    color: white;
    border-radius: 4px;
    white-space: nowrap; /* Evita que el texto se rompa en varias líneas */
}
```

13.4 Casos de Uso Prácticos

13.4.1 Configuración de Componentes de JavaScript

Es una excelente práctica para inicializar componentes (como carruseles, modales o galerías) sin acoplar la configuración al código JavaScript. Esto hace que el HTML sea más declarativo y fácil de entender.

HTML

```
<div class="carousel"
     data-velocidad="500"
     data-auto-play="true"
     data-items-visibles="3">
  </div>

<script>
const carousel = document.querySelector('.carousel');
// Leemos la configuración desde el dataset y la usamos para inicializar
const config = {
    // Es importante convertir los valores al tipo de dato correcto
    velocidad: parseInt(carousel.dataset.velocidad, 10),
    autoPlay: carousel.dataset.autoPlay === 'true',
    itemsVisibles: parseInt(carousel.dataset.itemsVisibles, 10)
};

// inicializarCarrusel(config);
</script>
```

13.4.2 Gestión de Estado de Elementos

Permiten rastrear el estado de un elemento (activo, inactivo, abierto, cerrado, contador) sin sobrecargar la lista de clases.

HTML

```
<button data-estado="inactivo"
        data-contador="0"
        onclick="actualizarEstado(this)">
  Clic aquí
```

```
</button>

<script>
function actualizarEstado(elemento) {
    let contador = parseInt(elemento.dataset.contador, 10) + 1;
    elemento.dataset.contador = contador;
    elemento.textContent = `Clics: ${contador}`;

    // Actualizamos el estado basándonos en la lógica
    if (contador > 5) {
        elemento.dataset.estado = 'activo';
    }
}
</script>
<style>
  [data-estado="inactivo"] { background-color: lightgray; }
  [data-estado="activo"] { background-color: lightgreen; }
</style>
```

13.4.3 Almacenamiento de Datos para Lógica de UI

Puedes guardar información relevante que no es visible pero que un script necesita para funcionar, como un ID de producto para una llamada a una API.

HTML

```
<button data-id-producto="12345" onclick="agregarAlCarrito(this)">
  Añadir al carrito
</button>

<script>
function agregarAlCarrito(boton) {
  const idProducto = boton.dataset.idProducto;
  console.log(`Añadiendo producto con ID: ${idProducto}`);
  // fetch('/api/carrito/agregar', { body: { id: idProducto } });
}
</script>
```

13.5 Mejores Prácticas

1. **Nombres Descriptivos**: Usa nombres claros (`data-user-id` en lugar de `data-uid`) y sigue una convención consistente (`kebab-case`). Esto mejora la legibilidad y el mantenimiento.
2. **Validación y Tipos de Datos**: Recuerda que `dataset` siempre devuelve **cadenas de texto**. Si almacenas números (`"42"`) o booleanos (`"true"`), deberás convertirlos manualmente (`parseInt()` , `parseFloat()` , `valor === 'true'`) antes de usarlos en tu lógica.
3. **Rendimiento**: Evita almacenar grandes cantidades de datos (JSONs extensos, ficheros codificados en base64) en `data attributes` . Estos datos aumentan el tamaño del DOM y pueden ralentizar el parseo y el rendimiento de la página. Para datos complejos, es mejor obtenerlos mediante una llamada a una API cuando sea necesario.
4. **Mantenibilidad**: Documenta el propósito de los `data attributes` que introduces, especialmente si su función no es obvia. Intenta que la lógica que los modifica esté centralizada y sea predecible.
5. **No son para datos visibles**: No uses `data attributes` para contenido que deba ser visible o indexable. Para eso están las etiquetas semánticas de HTML (`<p>` , `<h1>` , `<span>` , etc.). Los `data attributes` son para metadatos destinados a las máquinas (CSS/JS).
6. **Accesibilidad (A11y)**: Los `data attributes` **no son leídos por los lectores de pantalla**. Si necesitas comunicar información de estado o propiedades a tecnologías de asistencia, utiliza los atributos **ARIA** (`aria-label` , `aria-hidden` , `role` , etc.). `data-*` es para la lógica interna de la aplicación, `aria-*` es para la experiencia del usuario.

13.6 Ejercicios

13.6.1 Ejercicio 1: Atributos de Datos Simples

Crea un elemento `<div>` que represente a un usuario en una lista. Añádele dos atributos de datos:

- Un atributo para almacenar su ID de usuario, con el valor `951` .
- Un atributo para almacenar su estado, con el valor `activo` .

13.6.2 Ejercicio 2: Nombres de Atributos Compuestos

Crea un `<article>` para una entrada de blog. Necesitas almacenar información más compleja usando nombres de atributos compuestos:

- Un atributo para la **categoría principal**, con el valor `noticias`.
- Un atributo para la **fecha de publicación**, con el valor `2025-07-15`.

13.6.3 Ejercicio 3: Almacenar Datos de un Producto

Usa un único `<div>` para representar un producto en un catálogo de e-commerce. Almacena la siguiente información en sus `data attributes`:

- **ID del producto**: `prod-112`
- **Precio**: `49.99`
- **En stock**: `true`

13.6.4 Ejercicio 4: Configuración de un Componente

Prepara un `<div>` que funcionará como un carrusel de imágenes. Añádele los `data attributes` necesarios para una futura configuración con JavaScript:

- Un atributo `data-component` con el valor `carousel`.
- Un atributo `data-velocidad-animacion` con el valor `300`.
- Un atributo `data-navegacion-visible` con el valor `false`.

13.7 Soluciones

13.7.1 Solución Ejercicio 1

HTML

```html
<div data-id="951" data-estado="activo">
  Usuario 951
</div>
```

13.7.2 Solución Ejercicio 2

HTML

```html
<article
  data-categoria-principal="noticias"
  data-fecha-publicacion="2025-07-15">
  <h2>Título de la Noticia</h2>
  <p>Contenido del artículo...</p>
</article>
```

13.7.3 Solución Ejercicio 3

HTML

```html
<div
  data-id-producto="prod-112"
  data-precio="49.99"
  data-en-stock="true">
  <h3>Nombre del Producto</h3>
  <p>Descripción...</p>
</div>
```

13.7.4 Solución Ejercicio 4

HTML

```html
<div
  data-component="carousel"
  data-velocidad-animacion="300"
  data-navegacion-visible="false">
  </div>
```

13.8 Referencias y Recursos

13.8.1 Tutoriales y Guías

- MDN Web Docs: HTMLElement.dataset - La referencia oficial y más completa.
- CSS-Tricks - A Complete Guide to Data Attributes

14 Introducción a Web Components

Después de dominar HTML, el siguiente paso lógico es aprender a crear tus propios elementos HTML reutilizables. Los **Web Components** son un conjunto de tecnologías nativas del navegador que te permiten hacer exactamente eso: crear elementos HTML personalizados, encapsulados y reutilizables.

Dos de las piezas clave para construir estos componentes son las etiquetas `<template>` y `<slot>`.

14.1 Bloques de Construcción Nativos

14.1.1 El Elemento template: El Molde Oculto

Un elemento `<template>` es un mecanismo para guardar contenido HTML que **no se renderiza** cuando la página se carga. Su contenido está inerte y puede ser activado y clonado mediante JavaScript para usarse más tarde.

Piensa en él como un molde o una plantilla para una estructura HTML que planeas usar varias veces.

Sintaxis básica:

HTML

```html
<template id="plantilla-tarjeta">
  <style>
    /* Estilos encapsulados que solo aplican a esta tarjeta */
    .tarjeta {
      border: 1px solid #ccc;
      border-radius: 8px;
      padding: 16px;
      box-shadow: 2px 2px 8px rgba(0,0,0,0.1);
    }
  </style>
  <div class="tarjeta">
    <h2>Título de la Tarjeta</h2>
    <p>Contenido por defecto...</p>
  </div>
</template>
```

14.1.2 El Elemento `<slot>`: El Espacio para Personalizar

El elemento `<slot>` es un marcador de posición que se usa **dentro de un `<template>`**. Su función es "dejar un hueco" para que el usuario del componente pueda inyectar su propio contenido HTML desde fuera. Esto hace que los componentes sean flexibles y componibles.

Ampliando el ejemplo anterior:

HTML

```html
<template id="plantilla-tarjeta-con-slot">
  <style>
    /* ... estilos ... */
  </style>
  <div class="tarjeta">
    <h2><slot name="titulo">Título por Defecto</slot></h2>
    <div class="contenido">
      <slot name="contenido">Contenido por defecto...</slot>
    </div>
  </div>
</template>
```

Aquí hemos creado dos "huecos" (slots) con nombre: uno para el título y otro para el contenido.

14.2 Ejemplo Práctico: Creando un Elemento mi-tarjeta

Vamos a combinar todo para crear nuestro primer Web Component.

1. Definiendo el componente con JavaScript

El "cerebro" de un Web Component es una clase de JavaScript que define su comportamiento.

JAVASCRIPT

```javascript
// mi-tarjeta.js

// Obtenemos la plantilla que creamos en el HTML
const template = document.getElementById('plantilla-tarjeta-con-slot');

class MiTarjeta extends HTMLElement {
  constructor() {
    super();
    // Creamos un "Shadow DOM" para encapsular el contenido
    this.attachShadow({ mode: 'open' });
    // Clonamos el contenido de la plantilla y lo añadimos al Shadow DOM
    this.shadowRoot.appendChild(template.content.cloneNode(true));
  }
}
```

```
// Definimos el nuevo elemento personalizado para que el navegador lo reconozca
customElements.define('mi-tarjeta', MiTarjeta);
```

2. Usando nuestro nuevo elemento en HTML

Ahora podemos usar `<mi-tarjeta>` como si fuera cualquier otra etiqueta HTML. El contenido que pongamos dentro se insertará en los `<slot>` correspondientes gracias al atributo `slot`.

HTML

```
<script src="mi-tarjeta.js"></script>

<h1>Mis Componentes</h1>

<mi-tarjeta>
  <span slot="titulo">Tarjeta 1</span>
  <p slot="contenido">Este es el contenido personalizado para mi primera tarjeta. ¡Funciona!</
p>
</mi-tarjeta>

<mi-tarjeta>
  <span slot="titulo">Tarjeta 2</span>
  <ul slot="contenido">
    <li>Elemento de lista 1</li>
    <li>Elemento de lista 2</li>
  </ul>
</mi-tarjeta>
```

14.3 ¿Por Qué es Importante?

- **Encapsulación**: Los estilos y scripts dentro de un Web Component (en su Shadow DOM) no afectan al resto de la página, y viceversa. Se acabaron los conflictos de CSS.
- **Reusabilidad**: Puedes definir un componente una vez y usarlo en múltiples lugares de tu aplicación, o incluso en diferentes proyectos.
- **Nativo**: No necesitas librerías externas como React o Vue para usarlo. Es una característica estándar de los navegadores modernos. Entender los Web Components te dará una base sólida para entender cómo funcionan estas librerías por debajo.

14.4 Ejercicios

14.4.1 Ejercicio 1: Crear una Plantilla Básica

Crea una plantilla (`<template>`) para un "banner de alerta".

- La plantilla debe tener un `id` con el valor `plantilla-alerta`.
- Dentro de la plantilla, incluye un `<div>` con un `<h2>` que diga "Alerta" y un `<p>` con un texto genérico.

14.4.2 Ejercicio 2: Añadir Slots con Nombre a una Plantilla

Toma la plantilla del ejercicio anterior y hazla más flexible usando **`<slot>`**.

- Reemplaza el texto "Alerta" del `<h2>` por un `<slot>` con el `name="titulo-alerta"`.
- Reemplaza el párrafo genérico por un `<slot>` con el `name="contenido-alerta"`.

14.4.3 Ejercicio 3: Slots con Contenido por Defecto

Crea una plantilla para el perfil de un usuario (`<template>` con `id="plantilla-perfil"`).

- Incluye un `<img>` para la foto de perfil y un `<h3>` para el nombre.
- Ambos elementos deben estar dentro de **slots con nombre**.
- El `<slot>` para el nombre debe tener un texto por defecto que diga "Usuario Anónimo".
- El `<slot>` para la imagen no necesita contenido por defecto.

14.4.4 Ejercicio 4: Preparar el HTML para Usar un Componente

Imagina que ya existe un Web Component llamado `<producto-card>` que usa la siguiente plantilla. Escribe el código HTML que usarías en tu página para crear dos tarjetas de producto distintas, proporcionando el contenido para cada `slot`.

Plantilla (ya existe, no tienes que escribirla):

HTML

```html
<template id="plantilla-producto">
  ...
  <h2><slot name="nombre-producto"></slot></h2>
  <p><slot name="precio"></slot></p>
  <img src="..." alt="...">
  ...
</template>
```

Tu tarea: Escribe el HTML para usar `<producto-card>` dos veces con la siguiente información:

1. **Tarjeta 1**: Nombre "Laptop Pro", Precio "1200€".
2. **Tarjeta 2**: Nombre "Ratón Inalámbrico", Precio "25€".

14.5 Soluciones

14.5.1 Solución Ejercicio 1

HTML

```html
<template id="plantilla-alerta">
  <div>
    <h2>Alerta</h2>
    <p>Este es un mensaje de alerta genérico.</p>
  </div>
</template>
```

14.5.2 Solución Ejercicio 2

HTML

```html
<template id="plantilla-alerta">
  <div>
    <h2><slot name="titulo-alerta"></slot></h2>
    <p><slot name="contenido-alerta"></slot></p>
  </div>
</template>
```

14.5.3 Solución Ejercicio 3

HTML

```html
<template id="plantilla-perfil">
  <div>
    <slot name="foto-perfil"></slot>
    <h3><slot name="nombre-usuario">Usuario Anónimo</slot></h3>
  </div>
</template>
```

14.5.4 Solución Ejercicio 4

HTML

```html
<producto-card>
  <span slot="nombre-producto">Laptop Pro</span>
  <span slot="precio">1200€</span>
</producto-card>

<producto-card>
  <span slot="nombre-producto">Ratón Inalámbrico</span>
  <span slot="precio">25€</span>
</producto-card>
```

14.6 Referencias y Recursos

14.6.1 Documentación Oficial

- MDN Web Docs - Web Components: El portal principal de Mozilla para entender todas las tecnologías relacionadas con los Web Components.
- MDN - Elemento `<template>`: Documentación detallada sobre la etiqueta de plantilla.
- MDN - Elemento `<slot>`: Documentación específica sobre el elemento slot y cómo funciona.

14.6.2 Guías y Tutoriales

- web.dev - Custom Elements: Una guía práctica de Google sobre la creación de elementos personalizados.
- CSS-Tricks - A Guide to Web Components: Un artículo introductorio muy claro y bien explicado (en inglés).

15 Ejercicios Generales de HTML

Estos ejercicios te permitirán practicar la integración de todo lo aprendido en HTML hasta ahora. Recuerda que **no necesitas CSS ni JavaScript**, solo HTML semántico, estructurado y bien formado.

15.1 Ejercicios

15.1.1 Ejercicio 1: Página Personal Simple

Crea una página con la siguiente estructura:

- Un `<header>` que incluya tu nombre y una breve descripción.
- Un `<main>` con:
  - Un bloque de presentación con un párrafo sobre ti.
  - Una lista (ordenada o no) de tus hobbies.
  - Una tabla con tu horario semanal o tus comidas favoritas.
  - Una lista de enlaces a sitios web que te gusten.
- Un `<footer>` con:
  - Un enlace `mailto:` a tu correo.
  - La fecha actual como texto (escrita manualmente).

15.1.2 Ejercicio 2: Enciclopedia Temática

Crea una página que funcione como una mini enciclopedia sobre un tema de tu elección (ej: videojuegos, animales, mitología...).

- `<h1>` como título principal.
- Índice al inicio con enlaces internos a secciones usando `id` y `<a href="#id">`.
- Secciones `<h2>` y subsecciones `<h3>`.
- Uso de:
  - Listas de definiciones (`<dl>`, `<dt>`, `<dd>`).
  - Citas (`<q>` o `<blockquote>`).
  - Abreviaturas (`<abbr>`).
  - Datos importantes en `<mark>`, `<strong>`, etc.

15.1.3 Ejercicio 3: Catálogo de Productos

Diseña un catálogo simple de 3 productos:

- Cada producto en una sección con `id`.
- Índice navegable con enlaces internos.
- Tabla con nombre, descripción y precio.
- Uso de imágenes (`<img>`) con `alt` adecuado.
- Enlaces de contacto o más información ficticios.

15.1.4 Ejercicio 4: Formulario de Registro

Crea una página de inscripción a un evento ficticio:

- Un `<form>` con:
  - Campo de nombre (`<input type="text">`)
  - Correo electrónico (`<input type="email">`)
  - Selección (`<select>`) de tipo de asistente (estudiante, profesional…)
  - Casilla de aceptación de términos (`<input type="checkbox">`)
  - Botón para enviar (`<button>` o `input type="submit"`)
- Añade una tabla con fechas/lugares del evento.
- Descripciones organizadas con títulos y párrafos.

15.1.5 Ejercicio 5: Accesibilidad y Semántica

Crea una página accesible que cumpla:

- Uso de `<main>` , `<section>` , `<article>` , `<aside>` , `<footer>` , `<nav>` .
- Títulos con `<h1>` a `<h3>` en orden lógico.
- Una tabla con `caption` , encabezados con `<th scope="col">` , etc.
- Texto con énfasis semántico (`<em>` , `<strong>` , `<mark>`).
- Citas, abreviaturas, enlaces con texto descriptivo.

15.2 Soluciones Orientativas

> *Estas soluciones no son únicas. Se proponen como ejemplo de una estructura válida y correcta.*

15.2.1 Solución Ejercicio 1 (resumen):

HTML

```html
<header>
  <h1>Ana García</h1>
  <p>Estudiante de desarrollo web</p>
</header>
<main>
  <p>Me gusta aprender sobre diseño web y explorar nuevas tecnologías.</p>
  <ul>
    <li>Leer</li>
    <li>Escuchar música</li>
    <li>Salir a caminar</li>
  </ul>
  <table>
    <caption>Horarios Semanales</caption>
    <tr><th>Día</th><th>Actividad</th></tr>
    <tr><td>Lunes</td><td>Estudiar</td></tr>
    <tr><td>Martes</td><td>Leer</td></tr>
  </table>
  <p>Enlaces favoritos:
    <a href="https://developer.mozilla.org">MDN</a>,
    <a href="https://www.w3schools.com">W3Schools</a>
  </p>
</main>
<footer>
  <p>Contacto: <a href="mailto:ana@email.com">ana@email.com</a></p>
  <p>Julio 2025</p>
</footer>
```

15.2.2 Solución Ejercicio 2 (resumen):

HTML

```html
<h1>Enciclopedia de Astronomía</h1>
<nav>
  <ul>
    <li><a href="#planetas">Planetas</a></li>
    <li><a href="#estrellas">Estrellas</a></li>
  </ul>
</nav>
<section id="planetas">
  <h2>Planetas</h2>
  <dl>
    <dt>Júpiter</dt><dd>El planeta más grande del sistema solar.</dd>
  </dl>
  <blockquote cite="https://nasa.gov">
    <p>“Júpiter tiene más de 70 lunas.”</p>
  </blockquote>
</section>
<p><abbr title="National Aeronautics and Space Administration">NASA</abbr> estudia estos
cuerpos celestes.</p>
```

15.2.3 Solución Ejercicio 3 (resumen):

HTML

```html
<nav>
  <ul>
    <li><a href="#producto1">Zapatillas</a></li>
    <li><a href="#producto2">Camiseta</a></li>
  </ul>
</nav>
<section id="producto1">
  <h2>Zapatillas deportivas</h2>
  <img src="zapatillas.jpg" alt="Zapatillas negras con suela blanca">
  <table>
    <tr><th>Nombre</th><td>Zapatillas X</td></tr>
    <tr><th>Precio</th><td>59.99€</td></tr>
  </table>
</section>
```

15.2.4 Solución Ejercicio 4 (resumen):

HTML

```html
<h1>Registro al Evento Web 2025</h1>
<form>
  <label>Nombre: <input type="text" name="nombre"></label><br>
  <label>Email: <input type="email" name="email"></label><br>
  <label>Tipo:
    <select>
      <option>Estudiante</option>
      <option>Profesional</option>
    </select>
  </label><br>
  <label><input type="checkbox"> Acepto los términos</label><br>
  <input type="submit" value="Registrarse">
</form>
<table>
  <caption>Fechas y Lugares</caption>
  <tr><th>Ciudad</th><th>Fecha</th></tr>
  <tr><td>Madrid</td><td>10/09/2025</td></tr>
</table>
```

15.2.5 Solución Ejercicio 5 (resumen):

HTML

```html
<main>
  <section>
    <h1>Artículo sobre Accesibilidad</h1>
    <article>
      <h2>Importancia del HTML Semántico</h2>
      <p>El <strong>HTML semántico</strong> mejora la <mark>accesibilidad</mark>.</p>
      <p>Ejemplo: <abbr title="World Wide Web">WWW</abbr></p>
    </article>
    <aside>
      <p><q>La web debe ser para todos</q> - W3C</p>
    </aside>
    <table>
      <caption>Resumen de técnicas</caption>
      <tr><th scope="col">Elemento</th><th scope="col">Uso</th></tr>
      <tr><td><code>&lt;section&gt;</code></td><td>Divide el contenido en bloques temáticos</
td></tr>
    </table>
  </section>
</main>
```

15.3 Recursos

- MDN - Guía de HTML
- freeCodeCamp - Curso gratuito de HTML
- W3Schools - HTML Tutorial
- HTML Reference
- Frontend Practice (proyectos reales con HTML/CSS)